FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

IEC 61499 Compliant Web-based IDE for Function Block Design

Pedro da Cunha Teixeira e Faro Beirão

WORKING VERSION

Mestrado em Engenharia Informática e Computação

Supervisor: Prof. Rui Pinto

June 13, 2026

Resumo

Este documento ilustra o formato a usar em dissertações na Faculdade de Engenharia da Universidade to Porto (FEUP). São dados exemplos de margens, cabeçalhos, títulos, paginação, estilos de índices, etc. São ainda dados exemplos de formatação de citações, figuras e tabelas, equações, referências cruzadas, lista de referências e índices. Este documento não pretende exemplificar conteúdos a usar. É usado o *Loren Ipsum* para preencher a dissertação. Lorem ipsum dolor sit amet, FEUP consectetuer adipiscing elit. Etiam vitae quam sed mauris auctor porttitor. Mauris porta sem vitae arcu sagittis facilisis. Proin sodales risus sit amet arcu. Quisque eu pede eu elit pulvinar porttitor. Maecenas dignissim tincidunt dui. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Donec non augue sit amet nulla gravida rutrum. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Nunc at nunc. Etiam egestas. Donec malesuada pede eget nunc. Fusce porttitor felis eget mi mattis vestibulum FEUP. Pellentesque faucibus. Cras adipiscing dolor quis mi. Quisque sagittis, justo sed dapibus pharetra, lectus velit tincidunt eros, ac fermentum nulla velit vel sapien. Vestibulum sem mauris, hendrerit non, feugiat ac, varius ornare, lectus. Praesent urna tellus, euismod in, hendrerit sit amet, pretium vitae, nisi. Proin nisl sem, ultrices eget, faucibus a, feugiat non, purus. Etiam mi tortor, convallis quis, pharetra ut, consectetuer eu, orci. Vivamus aliquet. Aenean mollis fringilla erat. Vivamus mollis, purus at pellentesque faucibus, sapien lorem eleifend quam, mollis luctus mi purus in dui. Maecenas volutpat mauris eu lectus. Morbi vel risus et dolor bibendum malesuada. Donec feugiat tristique erat. Nam porta auctor mi. Nulla purus. Nam aliquam.

Abstract

Here goes the abstract written in English. It should say the same.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Sed vehicula lorem commodo dui Faculdade de Engenharia da Universidade to Porto (FEUP). Fusce mollis feugiat elit. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec eu quam. Aenean consectetuer odio quis nisi. Fusce molestie metus sed neque. Praesent nulla. Donec quis urna. Pellentesque hendrerit vulputate nunc. Donec id eros et leo ullamcorper placerat. Curabitur aliquam tellus et diam. Ut tortor. Morbi eget elit. Maecenas nec risus. Sed ultricies. Sed scelerisque libero faucibus sem FEUP. Nullam molestie leo quis tellus. Donec ipsum. Nulla lobortis purus pharetra turpis. Nulla laoreet, arcu nec hendrerit vulputate, tortor elit eleifend turpis, et aliquam leo metus in dolor. Praesent sed nulla. Mauris ac augue. Cras ac orci. Etiam sed urna eget nulla sodales venenatis. Donec faucibus ante eget dui. Nam magna. Suspendisse sollicitudin est et mi. Fusce sed ipsum vel velit imperdiet dictum. Sed nisi purus, dapibus ut, iaculis ac, placerat id, purus. Integer aliquet elementum libero. Phasellus facilisis leo eget elit. Nullam nisi magna, ornare at, aliquet et, porta id, odio. Sed volutpat tellus consectetuer ligula. Phasellus turpis augue, malesuada et, placerat fringilla, ornare nec, eros. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Vivamus ornare quam nec sem mattis vulputate. Nullam porta, diam nec porta mollis, orci leo condimentum sapien, quis venenatis mi dolor a metus. Nullam mollis. Aenean metus massa, pellentesque sit amet, sagittis eget, tincidunt in, arcu. Vestibulum porta laoreet tortor. Nullam mollis elit nec justo. In nulla ligula, pellentesque sit amet, consequat sed, faucibus id, velit. Fusce purus. Quisque sagittis urna at quam. Ut eu lacus. Maecenas tortor nibh, ultricies nec, vestibulum varius, egestas id, sapien. Donec hendrerit. Vivamus suscipit egestas nibh. In ornare leo ut massa. Donec nisi nisl, dignissim quis, faucibus a, bibendum ac, diam. Nam adipiscing hendrerit mi. Morbi ac nulla. Nullam id est ac nisi consectetuer commodo. Pellentesque aliquam massa sit amet tellus. Vivamus sodales aliquam leo.

Acknowledgements

I would like to express my gratitude to everyone that contributed to the realisation of this dissertation. I give my sincere thanks to:

My supervisor, Rui Pinto, for all confidence put onto me and the help throughout all the work.

Mariana Coimbra for her constant support, encouragement and feedback during this process. Her understanding and motivation were invaluable in the more demanding stages.

I would also like to thank my family and friends for their continuous support and encouragement, as well as everyone who contributed directly or indirectly to this project, whether through feedback, discussions, or assistance during testing and evaluation.

Pedro Beirão

"Programming is not a zero-sum game. Teaching something to a fellow programmer doesn't take it away from you. I'm happy to share what I can, because I'm in it for the love of programming."

John Carmack

Contents

List of Figures

List of Tables

List of Listings

List of Acronyms

CI	Continuous Integration
CPPS	Cyber-Physical Production System
CRDT	Conflict-free Replicated Data Type
DSL	Domain-Specific Language
FBD	Function Block Diagram
FEUP	Faculdade de Engenharia da Universidade to Porto
ICS	Industrial Control Systems
IDE	Integrated Development Environment
MEEC	Master in Electrical and Computer Engineering
PLC	Programmable Logic Controller
OT	Operational Transformation
RTE	Runtime Environment
SINF	Information Systems
SOA	Service-Oriented Architectures
SYSTEC	Research Center for Systems and Technologies
UI	User Interface

Chapter 1

Introduction

1.1 Context

Cyber-Physical Production Systems (CPPSs) [15] are complex systems that integrate physical processes with digital technologies to optimise production processes. Several software engineering approaches and technologies are utilised in the development and operation of CPPSs. One of such approaches is Function Block Diagram (FBD) [4], which are graphical programming languages used in industrial automation and control systems. FBD uses graphical function blocks to represent the logic and control of a system. The function blocks are connected in a network to form a control program. One of the benefits of FBD is that it provides a visual representation of the control logic, which can be easier to understand and debug than traditional textual programming languages. FBD is also modular, which means that function blocks can be reused in multiple programs, reducing development time and improving code maintainability. Also, FBD is often used in conjunction with Programmable Logic Controllers (PLCs) to control machinery and processes in manufacturing and other industrial applications [17, 19]. FBD programs are generally more complex to create and edit than traditional text-based languages, as they require specialised software tools that support graphical composition, connection management and deployment configuration. This additional complexity has been widely noted in industrial and academic contexts, particularly when compared to the relative simplicity of text-based programming workflows [6].

In industrial automation, it is common to use the *IEC 61499* standard, which defines an event-driven FBD approach for designing control software [22]. Unlike traditional PLC programming languages, *IEC 61499* emphasises modularity, reusability and explicit separation of events and data, allowing developers to design complex distributed systems in a scalable and maintainable way. Function blocks define both algorithms and interfaces for inputs, outputs and events making them ideal for CPPS development [8]. This approach is particularly suitable for distributed automation systems, where control logic must respond to events generated by multiple devices or sensors in real time. An example of such a function block interface is shown in Figure 1.1, highlighting the event inputs, outputs and data connections that enable flexible interaction between system components.

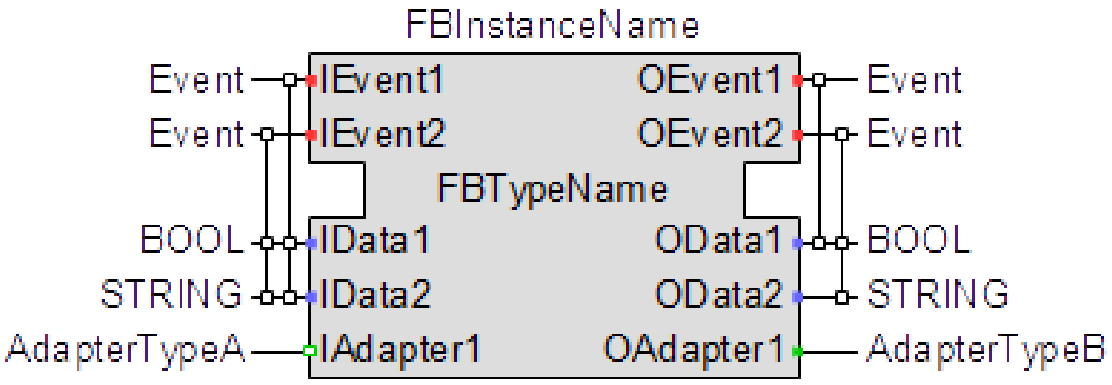


Figure 1.1: *IEC 61499* function block interface [10]

Runtime Environments (RTEs) provide the underlying execution layer for *IEC 61499* applications, responsible for running function blocks on target devices in accordance with the standard's event-driven execution model. In distributed CPPS environments, RTEs are deployed across multiple devices, enabling the execution of decentralised control logic while maintaining coordination between system components.

Conversely, Integrated Development Environments (IDEs) are responsible for the design, configuration and deployment of applications into CPPS infrastructures. In this role, they act as the primary interface between the engineer and the distributed runtime system, bridging the gap between high-level system modelling and low-level execution. IDEs typically provide a graphical User Interface (UI) like shown in Figure 1.2, with specialised tools for constructing function block networks, configuring device mappings and managing deployment workflows. Figure 1.3 represents the communications between these two layers.

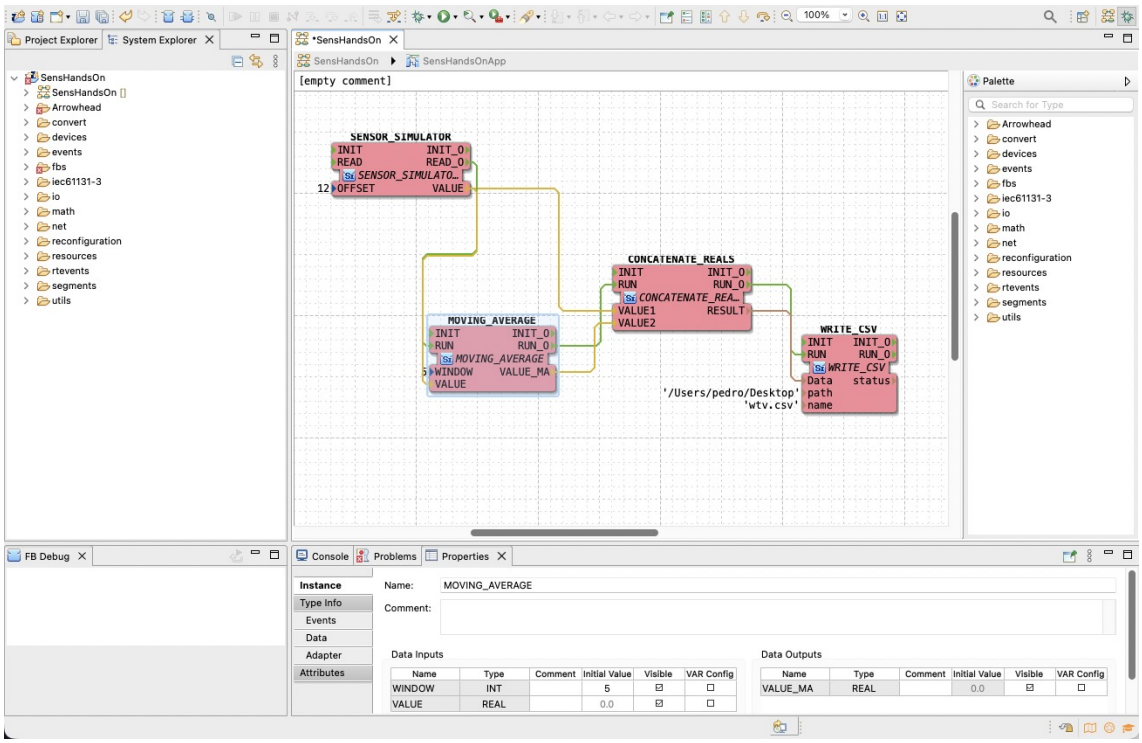


Figure 1.2: 4diac IDE [1]

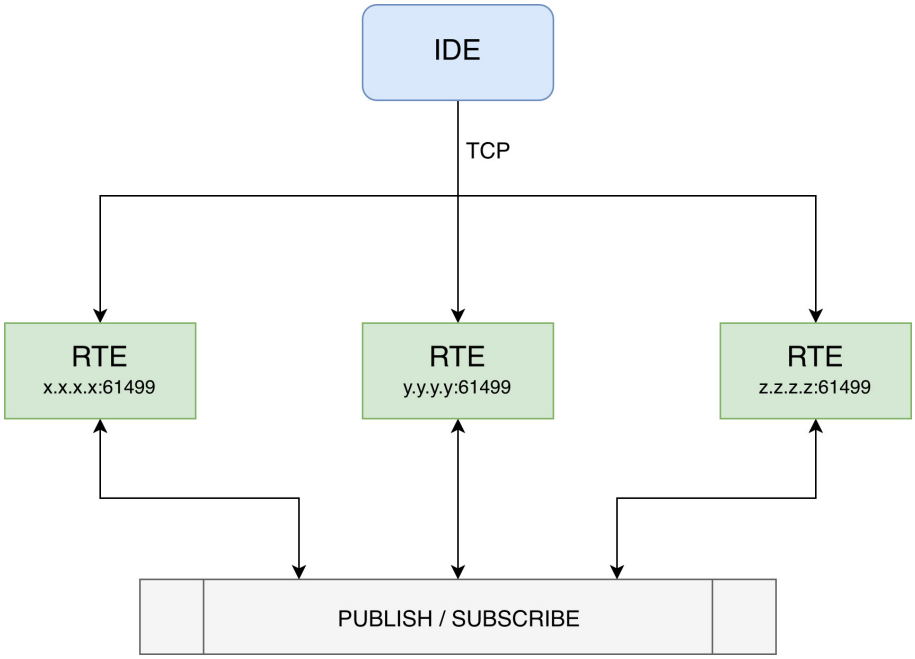


Figure 1.3: Communications between IDE and RTEs

1.2 Motivation

The evolution of CPPSs has increased the demand for flexible and intelligent control solutions capable of integrating physical processes with digital technologies. FBD and the *IEC 61499* standard offer a structured, modular approach to building such distributed automation systems. It provides not only a technically robust foundation for industrial automation, but also an elegant and structured way of reasoning about distributed control systems. To fully leverage these benefits, development tools must evolve alongside industrial needs, supporting more accessible and scalable workflows.

1.3 Problem

Existing *IEC 61499* development environments lack modern capabilities such as remote access, real-time collaboration and seamless integration with cloud infrastructures. These limitations hinder efficient development, maintenance and deployment of control applications within distributed CPPSs. As a result, current tools fall short in supporting the flexibility and scalability required by certain modern industrial automation systems. Besides this, the complexity of these current IDEs rules them out for many simpler use cases where the time wasted on setting up and learning them is not justifiable.

1.4 Research Questions

This research seeks to investigate how modern web and distribution technologies can be leveraged to enhance the development, deployment and operation of *IEC 61499*-compliant systems. This study is guided by the following main research question:

> How can a web-based IDE support the modelling, deployment and execution of *IEC 61499* applications?

To further refine the scope, the following sub-questions are proposed:

> To what extent can collaborative web technologies improve *IEC 61499* engineering workflows compared with traditional desktop IDEs?
>
> What are the performance and usability implications of adopting a web-based collaborative architecture for *IEC 61499* engineering?

1.5 Contributions

This dissertation contributes to the field of *IEC 61499* engineering by investigating the applicability of modern web technologies to the development of distributed automation systems. The main contributions of this work are:

- The design and implementation of *PTERO*, a web-based IDE for the development, deployment, and monitoring of *IEC 61499* applications.
- The development of an architecture that integrates modelling, function block management, deployment and monitoring within a single browser environment, reducing the fragmentation commonly found in existing workflows.
- The incorporation of real-time collaborative editing capabilities, enabling multiple users to work simultaneously on the same application.
- An analysis of the benefits and limitations of adopting a web-based collaborative architecture for *IEC 61499* engineering, providing insights for future research and development in this area.

Chapter 2

Background and Related Work

This chapter reviews prior work related to *IEC 61499*-based development and the used IDEs. The goal is to better understand the landscape, which will then allow the critical review of the state of the art. By providing a comprehensive analysis of the current landscape of development environments for CPPSs and the *IEC 61499* standard, we are able to categorise and critically examine the existing solutions, while looking for possible gaps in the ecosystem.

2.1 Methodology

A targeted literature review was carried out, focusing on studies addressing *IEC 61499* development tools, function blocks, CPPS and development environments, to then critically reflect upon their findings to help the writing of this dissertation and the planning of the final product.

2.1.1 Search and Selection

The search was conducted in digital libraries *ScienceDirect*, *IEEE Xplore*, *ACM Digital Library*, among others, as well as research paper aggregators like *Semantic Scholar* and *Scopus* using combinations of the following terms:

- *IEC 61499*
- FBD
- IDE
- CPPS

Following the search, a manual review process was conducted to refine and validate the results. Titles and abstracts were first screened to understand the relevance of the articles with respect to this dissertation. The publications were then examined in greater detail through full-text review, the reference lists of selected papers were manually inspected to identify relevant works that may not have been captured by the initial keyword search.

Articles mentioning new development environment proposals [22] were immediately included as their novel ideas were of great interest for a gap analysis on this ecosystem. These papers usually already contained a gap analysis of their own, which allowed for a literature review that was consistent with the prevalent opinion of the scientific community.

Furthermore, terms related to the state of the art that were discovered mid search were also included in some queries. The objectives of doing this were to get a better personal understanding of the scene, as well as to find interesting references that could be included in this dissertation to backup the written claims. Some of the terms used were:

- 4diac
- remote
- collaboration
- real-time

2.2 Distributed Function Block Engineering

Modern manufacturing and industrial automation are increasingly characterised by the transition from monolithic centralised control architectures towards distributed modular systems. The industry is moving towards a paradigm in which individual parts possess intelligence and communicate with one another. This shift is driven by the demands of Industry 4.0, which calls for flexibility and interoperability as key properties of the production infrastructure [7]. Monostori [15] provides a comprehensive analysis of CPPS, examining the theoretical foundations and practical implications of tightly coupling computational intelligence with physical manufacturing processes. The author explores how cyber-physical integration enables real-time monitoring, adaptive control and autonomous decision making in production systems, framing CPPS as a cornerstone of the fourth industrial revolution. This work establishes the concepts wording and architectural principles that help understand much of the research in this area and it serves as a reference for understanding why the flexible distributed control is necessary to realize the vision of CPPS.

The *IEC 61499* standard was developed precisely to address these requirements at the software level. This architecture defines a domain-specific modelling language for developing distributed Industrial Control Systems (ICS). It does so by extending and superseding the older *IEC 61131-3* standard, which was designed for centralised PLC-based control, providing next-generation systems with an architecture that enables distributed control [3]. The architecture encourages flexible reuse of components as well as reliable data exchange. Function blocks are interconnected into networks that form an application, which can then be distributed across multiple devices, a property that makes FBD a natural fit for CPPS deployments where hardware must collaborate to execute an unified control program. Torres, Gonçalves, and Pinto [24] conducted a literature review examining how *IEC 61499* can be leveraged specifically for CPPS development, mentioning possible enhancements with the use of a cloud-based Service-Oriented Architectures (SOA)

approach. Their paper examines the current state of adoption of the standard within industrial automation research, identifying recurring architectural patterns in CPPS implementations.

Conversely, Pinto et al. [19] presented an industrial case study showing that function block oriented approaches improve system modularity and reduce development effort compared to traditional procedural programming. Their study documents the migration of a production cell from a sequential control architecture to an *IEC 61499* function block model using the *DINASORE* RTE, using integration time as the main validation metric and demonstrating improved reconfigurability when production requirements changed. The results show that the encapsulation of behaviour and communication interfaces within function blocks leads to more maintainable and extensible software.

2.3 IEC 61499 Development Environments

Currently, the development ecosystem for *IEC 61499* is composed almost exclusively of desktop-based IDEs. These tools are typically monolithic applications installed locally on engineering workstations, requiring each member of a development team to independently install, configure and maintain the software on their own machine. The tools themselves tend to be resource intensive, which creates demands for the hardware, making them less accessible to users working on modest, shared or virtual machines. There is also the issue of platform dependency, as a niche and focused field, some IDEs are not crossplatform, forcing users to use virtual machines to be able to use them, further intensifying performance issues.

2.3.1 Open-Source

The most prominent open source implementation is the *4diac IDE* (based on the *Eclipse* framework) which supports a variety of RTEs such as *4diac FORTE* and *DINASORE*. This IDE is very feature rich at the cost of increased complexity, resulting in a steep learning curve, which makes it more difficult to use [27]. The *Eclipse* foundation on which it is built is itself a heavyweight platform and while this grants access to a broad plugin ecosystem, it also means that the IDE inherits the framework's architectural complexity. Navigating the interface to accomplish basic tasks such as creating a new function block type, designing the flow and deploying to a RTE can require substantial initial investment before productive work can begin.

Zoitl, Strasser, and Ebenhofer [29] demonstrated the use of *4diac IDE* to develop modular and reusable *IEC 61499* applications, highlighting the benefits of flexibility and adaptability in industrial settings. The paper provides a practical perspective on the strengths and limitations of this IDE.

An alternative is *FBME*, which is a much simpler modelling environment based on the *JetBrains MPS* language workbench. This IDE implements *IEC 61499* concepts through a set of integrated Domain-Specific Languages (DSLs). The main contribution of this work is the research on applying model-driven development principles to the architecture of the IDE itself [22], resulting in a very modular and extensible product.

2.3.2 Commercial

The commercial side is occupied by *EcoStruxure Automation Expert*, *ISaGRAF Workbench* [26] and other development environments that usually have their own built-in proprietary RTE. This tight coupling between the engineering tool and the runtime gives commercial vendors significant control over the end development experience, which typically results in polished, production-ready tooling with strong vendor support. However, it also introduces considerable lock-in: projects developed in one commercial environment are not easily portable to a different runtime or IDE and the proprietary nature of the underlying RTE limits the ability to inspect, extend, or adapt behaviour.

Commercial tools are also constrained by their licensing models. Per seat licensing is common, meaning that the cost of the toolchain scales directly with the size of the team and can not be viable for smaller organisations, academic research groups or early stage projects. This creates a tiered ecosystem in which well resourced industrial organisations have access to mature tooling, while smaller teams are restricted to the open source alternatives, each of which carries its own set of limitations as described above.

2.3.3 Local-first Development

All of the tools mentioned previously share a fundamental design decision: they play around a local-first development model, in which the project state lives on a single workstation. While this model has historically been the norm in industrial automation tooling, it introduces a set of limitations that become problematic as CPPS projects grow in scale and involve more distributed engineering teams.

The most immediate consequence is the absence of real-time collaboration. When two engineers need to work on the same *IEC 61499* application simultaneously, they must coordinate manually, typically by dividing the project into separate files or modules and merging their changes afterwards. These tools also require the IDE to maintain awareness of the deployment target's state. Without a shared backend, each engineer must independently configure their local environment to point to the correct runtime endpoints and therefore there is no single source of truth for the current deployment state of the system.

2.4 Collaborative Modelling

As projects grow in scale and involve distributed engineering teams, the question of how to coordinate concurrent modifications to a shared application model becomes central [11]. Version control through *Git* is inadequate in this domain as it operates on a line based difference algorithm designed for plain text source code. In the general case, it is not safe to merge structured model files using plain line based merge operations, as they can produce structurally consistent but semantically incorrect models . *IEC 61499* applications are stored as *XML* documents whose

semantic integrity depends on relationships between elements. A valid merge operation can still result in a broken application graph.

Approaches to real-time concurrent editing, Operational Transformation (OT) and Conflict-free Replicated Data Type (CRDT) offer a fundamentally different model of collaboration from the workflow of version control systems. Rather than requiring engineers to work in isolation and later reconcile divergent states, these techniques allow all participants to apply changes optimistically to their local replica, with the system guaranteeing that every replica converges to the same state. This shift ensures that inconsistencies are resolved continuously and automatically, rather than discovered only at integration time, enabling a concurrent engineering experience over shared structured artefacts such as *IEC 61499* application graphs.

2.5 Evolution of Web-Based Tools

The software engineering domain has seen a massive shift from desktop IDEs to cloud based environments. This trend is now influencing the domain of ICS and while recent studies have proposed a SOA to integrate *IEC 61499* with cloud layers [24], the engineering tools themselves remain still desktop bound.

In general software development, tools like *Visual Studio Code* have started offering web based counterparts to their desktop applications [25] in search of a more flexible user experience. Running entirely within the browser, these environments allow engineers to open, edit and navigate codebases from any device without installation. They integrate naturally with cloud hosted codebases and Continuous Integration (CI) pipelines. In fact, newer development tools like *Zed* make it a key goal of their products to have web platform support [28], treating browser delivery not as an afterthought but as a primary design constraint.

Web based editors are also often easier to extend with real-time collaboration because they operate within a network architecture, leverage standardised browser communication technologies such as WebSockets and naturally integrate with centralised synchronisation services. These characteristics reduce many of the distribution and interoperability challenges that desktop applications must address separately [21].

The move towards web based tooling is not limited to software development environments. Across a wide range of professional domains, browser-native applications have gradually replaced traditional desktop software. Collaborative document editors such as *Google Docs* allow multiple users to edit the same document simultaneously, with changes synchronised in real time. Similarly, design and prototyping platforms such as *Figma* have demonstrated that even complex graphical workflows can be delivered entirely through the browser while supporting collaboration and centralised asset management. Drawing tools like *Excalidraw* and *draw.io* further illustrate how browser based applications can provide sophisticated modelling and visualisation capabilities without requiring local installation, while providing live collaboration features that their desktop predecessors did not have.

A common characteristic of these platforms is that they treat collaboration as an architectural design rather than an addon. By maintaining a shared representation of the artefact on a central service, users can concurrently make changes with minimal configuration. This contrasts with many traditional desktop applications, where collaboration often relies on exchanging files with separate version control and synchronisation mechanisms. As a result, web based tools have created a new expectation on collaboration.

Some studies explore remote web based execution and editing of FBD. Rohat and Popescu [20] proposed an early approach to remote monitoring and interaction with *IEC 61499* applications through a web interface, identifying the practical benefits of browser access in distributed industrial settings. While the work predates many modern web technologies, it established the motivation for moving development and monitoring capabilities to the browser. More recently, Lyu et al. [14] revisited this direction by proposing fully remote cloud based platform for *IEC 61499* development, embedding *EcoStruxure AE* in a frontend web interface. This way they were able to move all the processing into a centralised cloud platform.

More generally, Sorokin and Vyatkin [23] proposes a new architecture for the development of web-based DSL tools, grazing on the field of of *IEC 61499* engineering by using *FBME* as an example of how it can be migrated into a distributed collaborative web environment. While this paper does not provide the groundwork and validation necessary for the development of specifically a *IEC 61499* tool, it does provide good general insights of how its architecture could be designed.

2.6 Comparative Analysis and Research Gap

Table 2.1: Comparison of IEC 61499 development environments and proposals.

Tool	Type	Platform	Remote deployment	Real-time collaboration	Comments
4diac IDE [29]	Open source	Desktop	×	×	Steep learning curve
FBME [22]	Open source	Desktop	×	×	Very modular
EcoStruxure AE	Commercial	Desktop	×	×	Proprietary ecosystem
ISaGRAF Workbench [26]	Commercial	Desktop	×	×	Licensing model
Rohat and Popescu [20]	Proposal	Web (monitoring only)	✓	×	No remote modelling capabilities
Lyu et al. [14]	Proposal	Web (embedded desktop IDE)	✓	×	Does not take advantage of web technologies for the modelling tool
Sorokin and Vyatkin [23]	Proposal	Web	✓	✓	Designs a general architecture for creating web DSL tooling
PTERO	New Proposal	Web	✓	✓	-

Current *IEC 61499* tools face significant limitations that hinder modern CPPS development. These gaps paint a picture of an ecosystem that has matured in terms of standards compliance, but has lagged considerably in adapting to the workflows and expectations of contemporary engineering teams. The following points describe the most significant gaps:

- **Platform Dependency:** Existing solutions are monolithic desktop applications, lacking the accessibility and portability that we have come to expect. Engineers are required to install and maintain native software on every machine from which they intend to work, creating friction at the start of projects and compounding over time as the team grows or its members change.
- **Absence of Real-Time Collaboration:** None of the existing tools provide support for real-time editing. Teams must rely on external version control systems and manual coordination to manage shared development, introducing overhead and increasing the risk of integration errors that would be avoided by a collaboration aware environment.
- **Lack of a Centralised System State:** Since tools are local-first, there is no single representation of a system's current state. This makes it difficult to maintain a consistent view of the system across a distributed team to integrate the development environment into automated deployment and monitoring pipelines.
- **Limited Accessibility:** Engineers cannot access a project without access to a fully configured workstation. This is a practical limitation in industrial settings, where on-site diagnosis of a running application may be necessary from a device in which no local tooling has been installed and the project is not set up.

Taken together, thse gaps suggest that the ecosystem is well positioned to benefit from a lightweight, browser-accessible development environment that prioritises collaboration, low setup overhead and integration with modern deployment workflows, without sacrificing the domain-specific features that *IEC 61499* engineers depend on.

Chapter 3

Proposed Solution

To address the limitations identified in existing *IEC 61499* development environments, this dissertation proposes the design of a *PTERO* (Programming Tool for Editing and Runtime Orchestration), a web-based IDE for modelling *IEC 61499*-compliant applications. The goal is to bridge the gap between modern collaborative software engineering practices and industrial automation tooling used in CPPSs.

Existing tools are predominantly desktop based and poorly suited for distributed development. In contrast, the proposed solution adopts a centralised web-based architecture that enables platform independence, simplified setup, simplified deployment and real-time collaboration.

The main reason to propose using web technologies is to solve the issues that affects any local-first development environment. By having the IDE be hosted by a centralised server, the following key aspects are achieved:

- **Ease of setup**. Instead of each team member having to install a resource heavy program and set it up with the team's code and configurations, a single server is set up and the team members simply have to connect to it via their browser.
- **Ease of deployment**. Only the server needs to be aware of all of the target RTEs and have connection to them. When a team member wants to deploy, the request will be routed through the server, acting as the middleware, removing the need for each development machine to have connection with the target RTEs.
- **Real-time collaboration**. All data is kept and directly modified in the server itself, instead of requiring a periodic synchronisation system like *git*. FBD coded programs, since they are meant for visual editing, need to be stored in formats that don't easily handle merge conflicts. Real-time combats this by keeping track of atomic changes, instead of blocks of text.

3.1 Approach

The approach is derived from functional and non-functional requirements extracted from the *IEC 61499* standard, an analysis of existing tools and the gaps identified in the state of the art. Functional requirements include:

- Creation of function block types
- Design of the event and data flow
- Deployment to compliant RTEs
- Real-time collaboration

Non-functional requirements should ensure that the solution does not reflect negatively when compared with *4diac IDE* in some key aspects as well as bridge the gaps identified. These include:

- Platform independence
- Reduced setup complexity
- Improved usability for distributed teams
- Ease of extensibility for future improvements
- The webpage should load in less than 2 seconds
- Deployment should take just as long
- Should be able to handle 100 users concurrently

The effectiveness of the approach is assessed through the implementation of a functional prototype and its direct comparison with current established IDEs, focusing on usability, accessibility and workflow efficiency. This evaluation demonstrates the feasibility of applying web based principles to *IEC 61499* development and their potential benefits for distributed CPPSs.

3.2 System Architecture

The proposed system follows a centralised server architecture designed to support distributed CPPS development. Instead of the IDEs having direct communication with the RTEs, the server acts as an intermediary, meaning all requests go through it and removing the need for direct device-level configuration on individual machines. Besides serving the frontend to the users, it also stores all data in a centralised database. The development pipeline therefore consists of the following three core components, organised as displayed in Figure 3.1:

- **Frontend IDE**: A web interface for setting up and modelling the *IEC 61499* applications.

- **Server Middleware**: A centralised backend responsible for state management, collaboration and communication.
- **RTE**s: External execution environments where the applications are deployed.

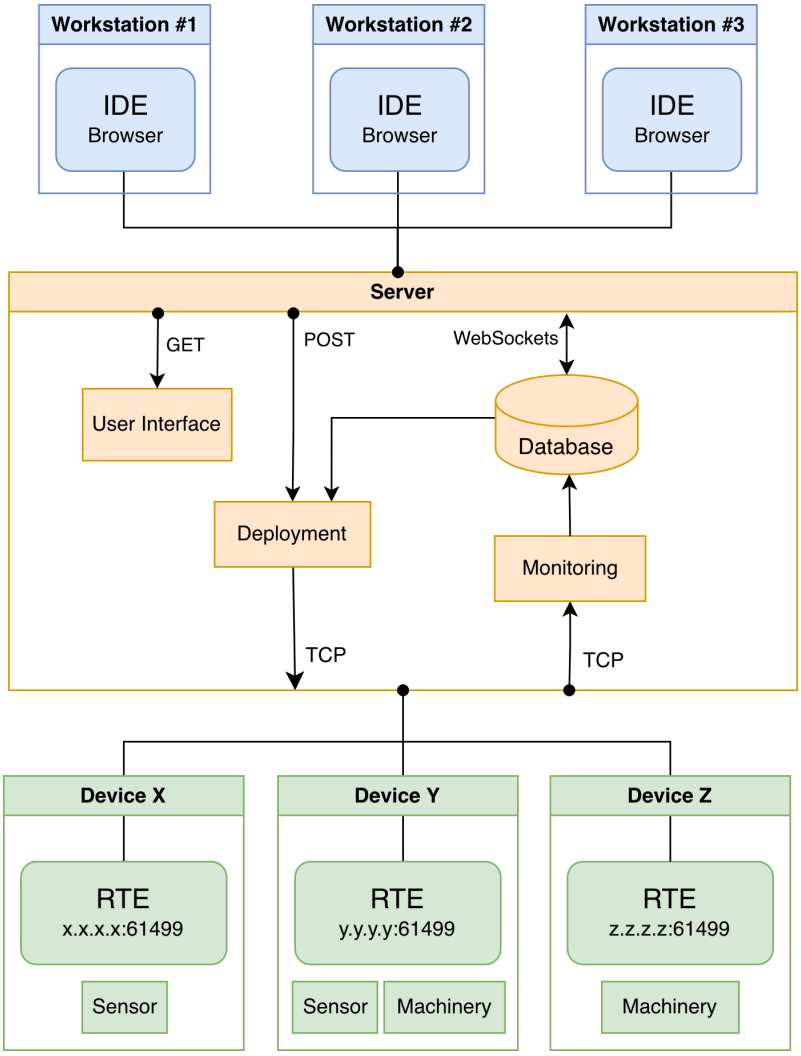


Figure 3.1: Communication through the server

Being the middleware, there are 4 different types of network communications that the server will need to handle, each with it's own requirements:

- **Serve webpage**: Communicates via *GET* requests to deliver the *html*, *css* and *js* files that make up the IDE interface.
- **Real-Time Collaboration**: Frequent atomic updates whenever changes are made.
- **Requested deployment**: A team member sends a *POST* request, that tells the server to deploy.

- **Deployment sockets**: Communicates via *TCP* with each RTE to send commands and monitor.

3.3 Real-Time Collaboration Model

A key requirement of the proposed system is support for concurrent editing of structured *IEC 61499* programs. Real-time collaborative systems shift conflict management from the merge phase to the editing phase itself, all participants may operate on the shared artefact simultaneously, with the system responsible for converging divergent states automatically. To address this, the system adopts a CRDT approach, enabling multiple users to edit shared application models simultaneously. Changes are propagated in real time and merged deterministically without conflicts. Compared to OT approaches, CRDTs provide a more suitable foundation for non linear structured data. Where in OT a semantic state change needs to be expressed as a textual change at a specific offset, in CRDT it keeps the semantic meaning [12].

The graph based structured data that in other IDEs would be stored as a conflict prone file in formats such as *XML*, should instead be kept as individual fields. A change in a single element should prompt the CRDT to only look into that specific field and not the document as a whole. The class diagram in Figure 3.2 illustrates how the fields can be organised.

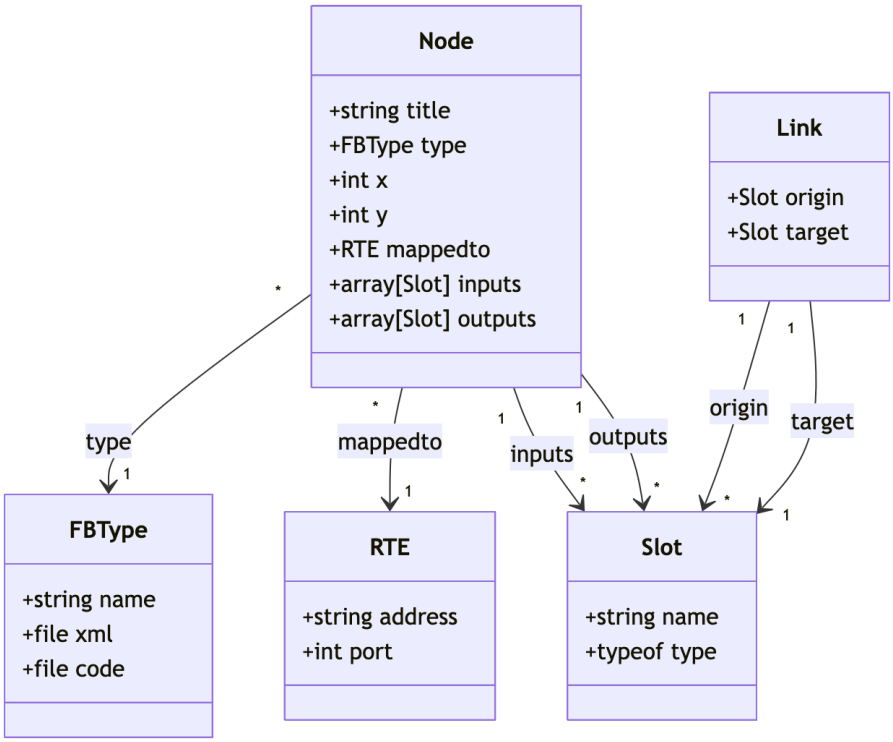


Figure 3.2: Class diagram of the program's data

3.4 Deployment Model

Deployment in the proposed system is based on a centralised orchestration model. The server translates the graphical application model into a sequence of deployment commands that are executed on target RTEs. This approach ensures consistent deployment across distributed environments while abstracting low level communication from the user. The deployment process follows the following conceptual stages:

1. Synchronising the function block types across devices
2. Sending execution commands to the assigned RTEs
3. Start monitoring processes

3.5 Methods

This section describes the research methodology adopted to address the research questions defined in this dissertation. Given the practical and design-oriented nature of the problem, the methodology focuses on the design, implementation and evaluation of a piece of software. The chosen methods are appropriate for investigating how web-based technologies can enhance *IEC 61499* development environments, as they allow both the realisation of a concrete solution and the systematic assessment of its capabilities relative to existing tools.

3.5.1 Research Design

This research follows a design-oriented research approach. The core of the research consists of the design and implementation of a functional prototype of the solution. Rather than aiming for statistical generalisation, the study focuses on demonstrating feasibility and identifying qualitative improvements over existing environments. The prototype serves as a research artefact used to explore architectural decisions, usability considerations and workflow implications.

3.5.2 Evaluation

Evaluation is conducted through qualitative analysis and comparative assessment against established *IEC 61499* IDEs. Criteria such as accessibility, usability, feature coverage and support for collaborative workflows are used to assess how effectively the proposed solution addresses the limitations identified in the literature and state of the art.

Chapter 4

Implementation

To be able to prove the proposed solution is viable and will successfully address the gap identified, a prototype needs to be developed and afterwards validated. This chapter describes the technical implementation of the proposed system, detailing the server architecture, frontend implementation, collaboration system and deployment mechanism used to realise the conceptual design presented in Chapter 3.

4.1 Runtime Environment

Out of all RTEs compatible with *IEC 61499*, this prototype will focus on supporting *DINASORE* (Dynamic INtelligent Architecture for Software and MOdular REconfiguration) [18]. It is much less complex than others, making it ideal for a proof of concept IDE. As an open-source project, *DINASORE* can be freely inspected and studied, which is particularly valuable in a research and prototyping context where flexibility is prioritised over real-time performance guarantees. This ease of study rules out the use of commercial RTEs like *EcoStruxure Automation Expert's "dPAC"* and *ISaGRAF Runtime*. Other open-source RTEs were considered, but none were as well-suited as *DINASORE*. *4diac FORTE* for example was way too extensive and complicated to be able to easily study its source code. Figure 4.1 represents the architecture of this RTE where *PTERO* aims to replace the *4DIAC Interface*.

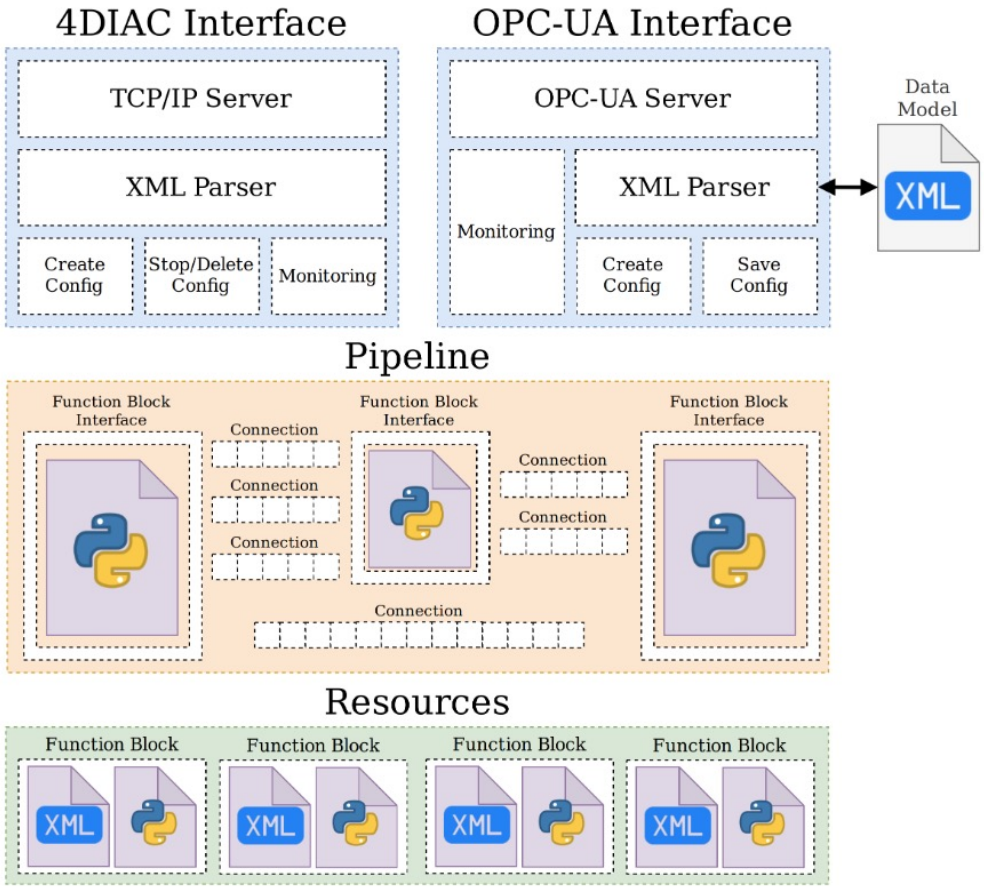


Figure 4.1: *DINASORE*'s architecture [18].

By default, *DINASORE* uses the port *61499* to receive commands from the IDE and send responses. This is important to note as it is the port that will be used as example in this chapter. The exact details of the communication implementation will be explained further in Section 4.5.

4.2 Server

The design begins by setting up a server that will serve the IDE interface as a webpage, store and manage all the data and handle communication with the RTEs. Taking into account the features we plan on implementing, *JavaScript* running on *Node.js* were chosen due to the extensive library ecosystem. For real-time collaboration we will need a library to run in both the server and the frontend, since *JavaScript* runs natively in the browser, it further reinforces that it should also be used in the server. The library choice will be explained in depth in Section 4.4, proving the previous train of thought.

4.3 Frontend

The frontend is the UI that will be used to create and modify the FBD programs.

Each subsection will first explain and describe what an IDE should implement and follow by showing how the prototype solved the proposed requirements.

4.3.1 Devices Configuration View

In the interest of distributed CPPSs, *IEC 61499* allows function blocks to be mapped to different *DINASOREs*, so we need a clean way to define the resources that are available. Both the *DINASORE* port and the *SSH* port need to be defined for each defined since deployment communication will be made through both.

Each device should have several fields that the team can configure:

- **Name**: Human friendly name to quickly identify what device this is
- **Address**: The address of the device
- **DINASORE port**: The port where *DINASORE* is running
- **Color**: The color associated with this *DINASORE* instance, providing a visual distinction in the *Graph View* between function blocks mapped to different instances.
- **SSH port**: The open *SSH* port of the device.
- **SSH user** and **SSH password**: The *SSH* login credentials.
- **SSH path to DINASORE**: Path in the *SSH* filesystem pointing to where *DINASORE* is located.

With the existence of the middleware, this solution removes the requirement for each team member to configure the available *DINASORE* instances in their own IDE and instead keeps a shared configuration in the server database.

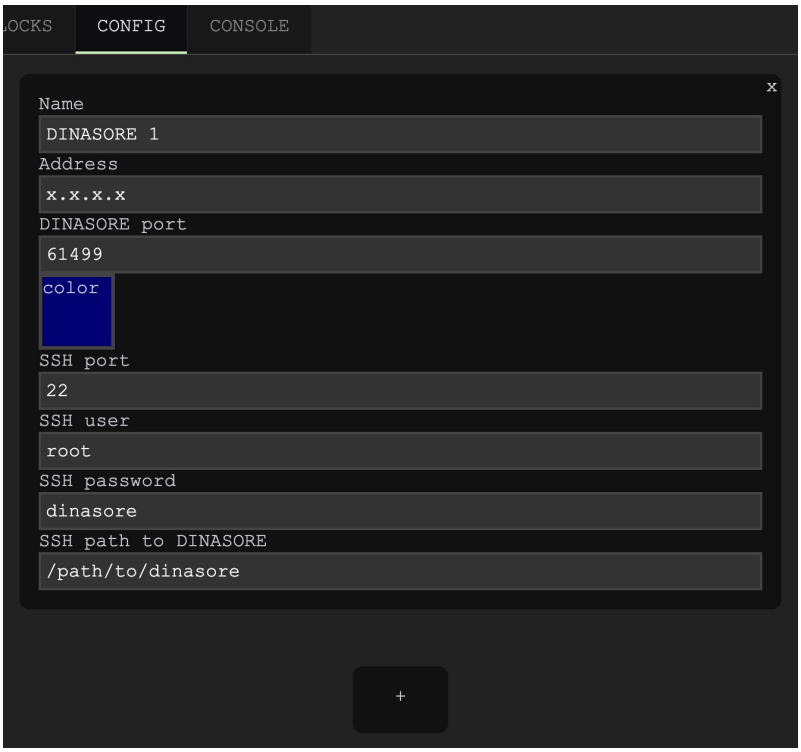


Figure 4.2: Devices Configuration View

4.3.2 Function Blocks Editor View

This is where individual function blocks are defined. It allows the team to specify the interface of each function block, its input and output data ports and events, as well as program the internal behaviour. This separation of interface and implementation follows the *IEC 61499* model of encapsulation, where a function block exposes a well defined interface while keeping its internal logic self-contained.

Like in the other RTEs, in *DINASORE* the interface of the function block is defined in a *xml* format file with a *.fbt* extension, which results in a function block that can be displayed graphically like seen in Figure 4.3. The internal behaviour of the function blocks in *DINASORE* is programmed in *Python*.

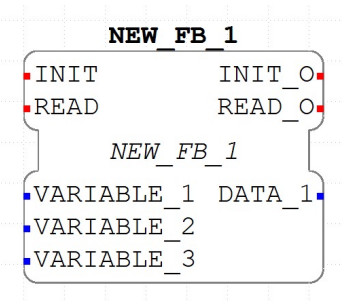


Figure 4.3: Example function block defined via *xml*

For the IDEs to be able to detect and use the function block, both the *.fbt* file and the *.py* file need to be placed in a specific folder, right next to each other, like shown in Figure 4.4. In this prototype this is handled automatically in the server database, removing the need to manually move files like it is needed in *4diac IDE*.

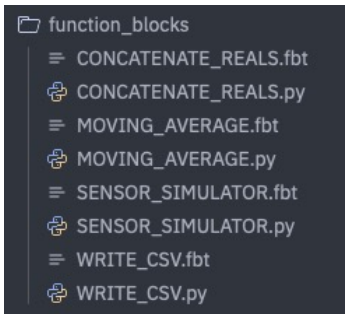


Figure 4.4: Filesystem folder with various function block files

This view contains a split view text editor: a side for the *.fbt* file and the other for the *.py* file. It would be possible to create a visual editor for designing the interface, instead of having to write *xml* by hand, but the added complexity would not be justified at this prototyping stage. The *xml* format is sufficiently structured that a text editor provides an adequate experience and a dedicated visual interface remains a natural direction for future development.

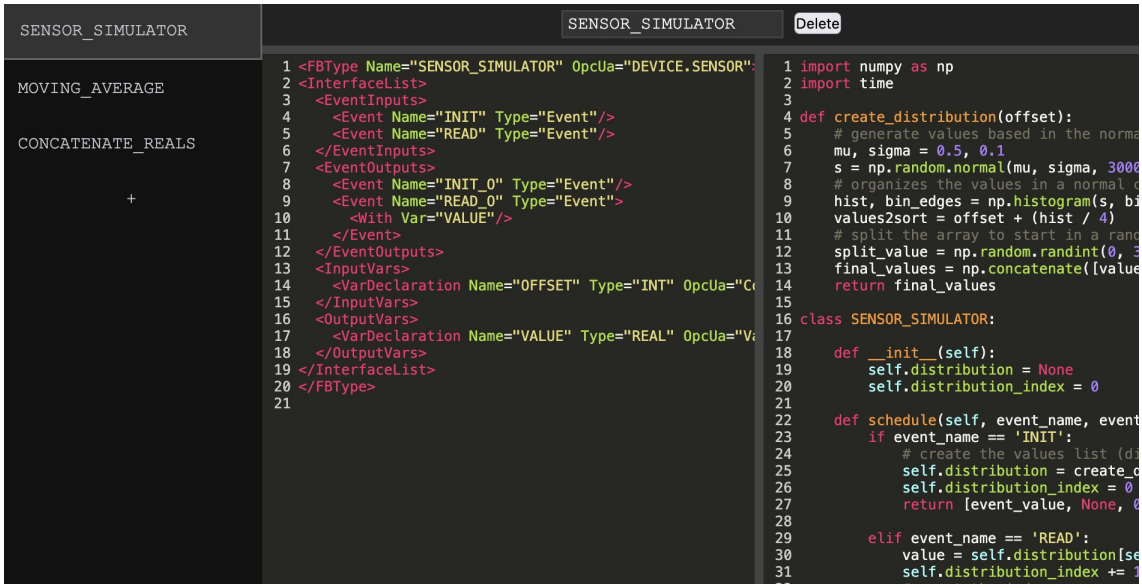


Figure 4.5: Function Blocks Editor View

4.3.3 Graph View

This view is the primary workspace for composing the application. Function blocks are placed on the canvas and connected together by drawing connections between their data and event ports,

forming the application network. Event ports are commonly color coded red, while data ports are color coded blue.

In *DINASORE*, all *INIT* event ports are by default assigned to *EMB_RES*, the default resource that sends an event when the application is started. This removes the need to connect the *INIT* event ports, but the *READ* event port is not assigned by default and therefore needs to be connected in every function block. The `schedule()` function will only be called and generate output when an event is sent to the *READ* event port. The described event driven execution model is what allows an explicit specification of the execution order of function blocks. Periodic out of event execution of function blocks is also possible with the implementation of a looping function block that triggers events [10].

A function block will have some input and output data ports, each with its own associated data type. For example, an *INT* input port will not accept data from a *WORD* output port and therefore should not be able to connect to it. It is also possible to set a default value for an input if no output is connected to it, there is an editable string field for this which gets converted to the expected data type by the RTE. The data types supported by *IEC 61499* are derived from and defined by *IEC 61131-3* as listed in Table 4.1.

Type	**Description**	**Size**
`BOOL`	Boolean	1-bit
`SINT`	Short integer	8-bit
`INT`	Integer	16-bit
`DINT`	Double integer	32-bit
`LINT`	Long integer	64-bit
`REAL`	Floating point	32-bit
`LREAL`	Long floating point	64-bit
`STRING`	ASCII string	Variable
`WSTRING`	Unicode string	Variable
`TIME`	Time interval	
`DATE`	Date	

Table 4.1: IEC 61131-3 Data Types [9]

Each function block can also be mapped to a *DINASORE* instance, with the color coding defined in the Devices Configuration View providing an immediate visual indication of how the application is distributed across the available devices.

This system was achieved in the prototype with the use of the *LiteGraph.js* library, a graph node editor for the browser. A lot of other libraries were considered, but this one was chosen due to the requirements of the proposed solution. Let us discuss the alternatives that were considered to understand why this choice is important:

- **Rete.js**: A very flexible and complete editor framework, but the added complexity makes it harder to integrate the desired features. It is also a more general purpose editor, meaning

the multi-input and multi-output function blocks with specific data types are not as well supported.

- **xyflow**: A highly customizable framework composed of React Flow and Svelte Flow. While it provides excellent visual customisation and integration with frontend frameworks, it suffers from the same problems as *Rete.js*.
- **Drawflow**: A lightweight and simplistic library with a focus on data flow execution. Although simple to integrate, it lacks many features required for designing *IEC 61499* function blocks, such as robust type handling and multi-input and multi-output.
- **Baklava.js**: With a clean architecture and flexibility, this would be a good alternative, but since it specifically targets *Vue* and requires a more intricate setup, it was not the most appropriate choice for a prototype.
- **GoJS**: Very powerful commercial alternative. It offers advanced functionality and flexibility, but its licensing model make it less suitable for lightweight research prototypes and open development environments.
- **Node-RED**: A flow based programming environment. While conceptually similar, its architecture is more focused on runtime execution than on being a visual editor of structured data.

Among these alternatives, *LiteGraph.js* presented the best balance between simplicity, flexibility and ease of integration. Its builtin support for typed inputs and outputs, lightweight architecture and direct focus on visual data flow programming made it especially suitable for the requirements of the proposed solution. Even so, it was necessary to implement custom behaviour on top of some functions of *LiteGraph.js* to achieve the desired behaviour. The choice of the graph library is very important, since the requirements of *IEC 61499* differ significantly from those of most conventional graph editors, as such, special care must be taken to ensure that these requirements are properly supported.

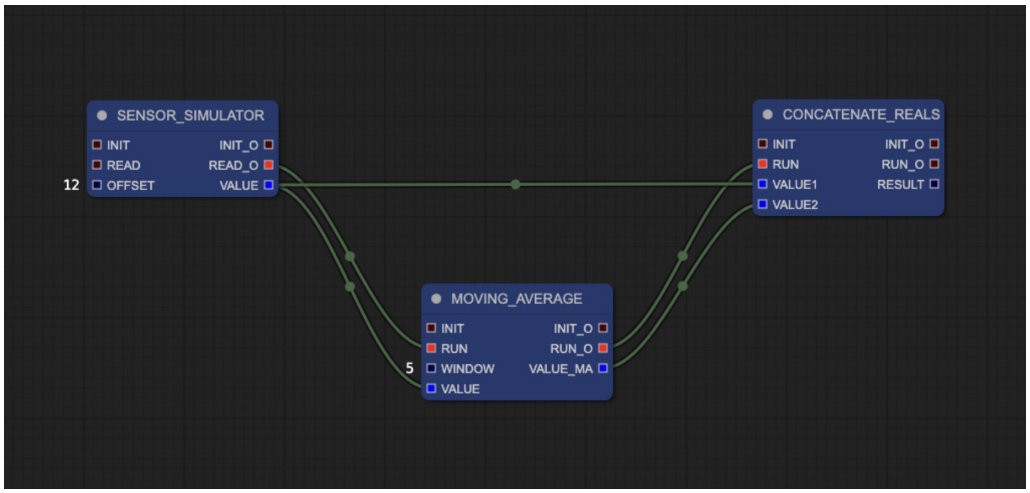


Figure 4.6: Graph Editor View

4.3.4 Console View

This view will display the console logs that are generated when deploying. The contents of it will be discussed in detail in Section 4.5.

```
python3 ./sync/synchronize.py
Starting copying process
Copying process completed
Synchronized 1 DINASOREs
<Request Action="QUERY" ID="0"><FB Name="*" Type="*"/></Request>
<Response ID="0" />
<Request Action="CREATE" ID="1"><FB Name="EMB_RES" Type="EMB_RES"/></Request>
<Response ID="1" />
<Request Action="CREATE" ID="2"><FB Name="SENSOR_SIMULATOR" Type="SENSOR_SIMULATOR"/></Request>
<Response ID="2" />
<Request Action="WRITE" ID="3"><Connection Destination="SENSOR_SIMULATOR.OFFSET" Source="12"/></Request>
<Response ID="3" />
<Request Action="CREATE" ID="4"><FB Name="MOVING_AVERAGE" Type="MOVING_AVERAGE"/></Request>
<Response ID="4" />
<Request Action="WRITE" ID="5"><Connection Destination="MOVING_AVERAGE.WINDOW" Source="5"/></Request>
<Response ID="5" />
<Request Action="CREATE" ID="6"><FB Name="CONCATENATE_REALS" Type="CONCATENATE_REALS"/></Request>
<Response ID="6" />
<Request Action="CREATE" ID="7"><Connection Destination="MOVING_AVERAGE.VALUE" Source="SENSOR_SIMULATOR.VALUE"/></Request>
<Response ID="7" />
<Request Action="CREATE" ID="8"><Connection Destination="CONCATENATE_REALS.VALUE2" Source="MOVING_AVERAGE.VALUE_MA"/></Request>
<Response ID="8" />
<Request Action="CREATE" ID="9"><Connection Destination="CONCATENATE_REALS.RUN" Source="MOVING_AVERAGE.RUN_O"/></Request>
<Response ID="9" />
<Request Action="CREATE" ID="10"><Connection Destination="MOVING_AVERAGE.RUN" Source="SENSOR_SIMULATOR.READ_O"/></Request>
<Response ID="10" />
<Request Action="CREATE" ID="11"><Connection Destination="CONCATENATE_REALS.VALUE1" Source="SENSOR_SIMULATOR.VALUE"/></Request>
<Response ID="11" />
<Request Action="START" ID="12"/>
<Response ID="12" />
```

Figure 4.7: Console View

4.4 Real-Time Collaboration

In *PTERO* the collaborative editing system was implemented using *Yjs* [16], a CRDT framework designed for distributed collaborative applications, allowing users to concurrently edit shared data structures while automatically resolving conflicts in a deterministic manner. *Yjs* was chosen in detriment of alternative CRDT implementations like *Automerge* due to its large ecosystem. With built-in support for many text editor libraries [5], such as *CodeMirror*, adding real-time collaboration to the *Function Blocks Editor View* is very straightforward.

The graph structure representing the FBD application is synchronised through shared *Yjs* data structures. Operations such as creating function blocks, deleting connections, modifying inputs or moving nodes are propagated in real time to all connected clients. Each user therefore observes the same graph state with minimal delay. To integrate collaboration into this editor, the internal state changes of the graph were connected to the shared *Yjs* document. Whenever a local modification occurs in the graph, the corresponding operation is serialized and applied to the shared collaborative state. Likewise, remote updates received from other users are reflected back into the local graph editor instance.

4.5 Deployment

The deployment process is responsible for taking the application defined in the *Graph View* and bringing it to life across the configured *DINASORE* instances. It is broken down into a sequence of steps, each of which is described in the following subsections.

4.5.1 Synchronise with DINASOREs

The first step is to ensure that each *DINASORE* instance has access to the function blocks that it will need to run the application. This is done by sending all the *.fbt* and corresponding *.py* files in the database, to every device. To achieve this we can use the `synchronize.py` script that accompanies the *DINASORE* source code. A configuration file needs to be provided that tells the script where to send the function blocks to, like shown in Listing A.3. The server must have a valid *SSH* connection with each device in order to connect and send the files.

4.5.2 Send Commands

Once the function block files are in place, the server will send the necessary commands to each *DINASORE* instance to build and start the application. This includes creating the function block instances, writing the initial input values, establishing the connections between event and data ports and finally sending the start command to begin execution. The order in which these messages are issued is important, as connections can only be established between existing nodes. The following two subsections describe how the messages are constructed and what they contain.

4.5.2.1 Build messages

Before the messages can be sent, they need to be built into the format that *DINASORE* is expecting. It consists of a two part message: the first part refers to the configuration name, which by default we will use *EMB_RES*, while the second is the actual message payload containing the wanted command. Each of these parts are preceded by a 3-byte header composed of a `0x50` byte followed by 2 bytes indicating the size of the corresponding content. A representation of this format is presented in Figure 4.8.

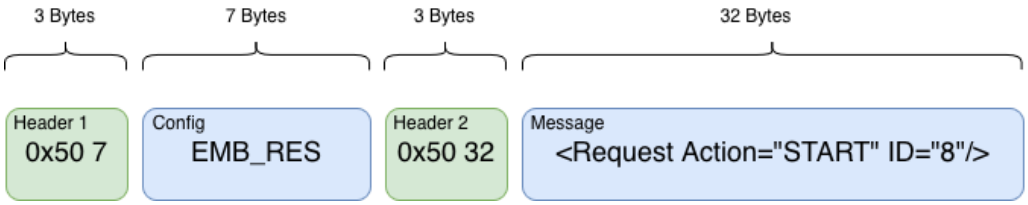


Figure 4.8: *DINASORE* communication format

The *4diac_simulator.py* file, part of the *tests/* folder of *DINASORE*, implements this method in *Python* as shown in Listing A.4. For this prototype, it was necessary to implement it in *JavaScript*, as shown in Listing A.5.

4.5.2.2 Send Messages

Now that we have defined how to build the messages, we can explore the sequence of commands that need to be sent to each *DINASORE* instance. The server connects to each instance and transmits these messages sequentially. The order of message transmission is critical, as connections can only be established between existing nodes. The sequence is represented in Figure 4.9. A more technical and in depth analysis of the messages sent can be found in Section A.4.

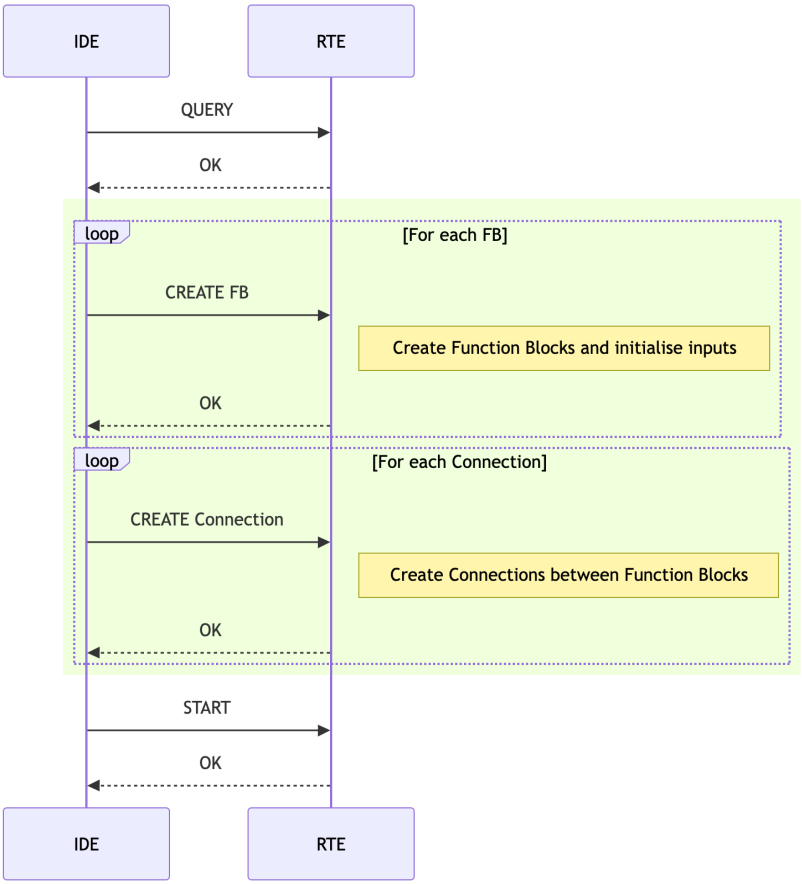


Figure 4.9: Overview of the communication sequence when deploying

4.5.3 Console

To give the team visibility into the deployment process right from the IDE, without having to log into the server, a console is provided in the *Console View* that displays a live log of each action taken during deployment. There are two main things being logged:

- **Synchronise**: The output of running the *Python* script is displayed so that the team see that the *SSH* communication is being done properly.
- **Send Commands**: As commands are sent and responses are received, they are displayed, allowing the team to make sure each command is formulated correctly, being processed and a response is issued.

4.5.4 Watch

Once the application is running, the values of the node outputs are continuously retrieved from the *DINASORE* instances and displayed in the *Graph View*, providing immediate feedback on the state of the running application. This is particularly useful for debugging and validating the behaviour of the application during development. In order to get these values, the IDE must tell the *DINASORE* instances which output slots it wants to watch and then request the values. In this prototype, for simplicity, all data outputs are watched and we need to send a watch creation request for each.

Figure 4.10 represents the sequence that the messages need to be sent. Details about these requests and the associated responses can be found in Section A.5. When the watch values are retrieved, they are displayed in the right side of the corresponding output slots, demonstrated in Figure 4.11.

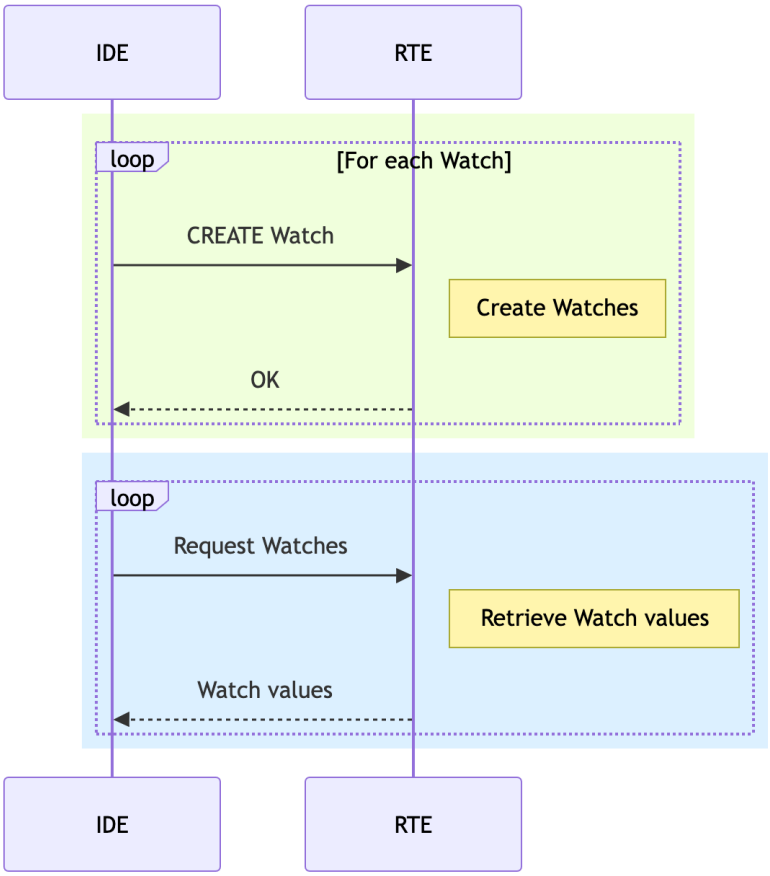


Figure 4.10: Overview of the communication sequence when watching

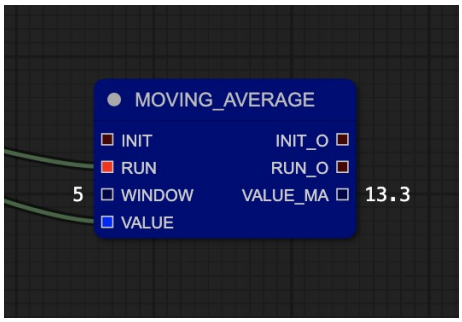


Figure 4.11: Watching an output slot in the *Graph View*

4.6 Distribution

Chapter 5

Validation

The purpose of this chapter is to evaluate whether the proposed solution achieves the proposed objectives, effectively addressing the gap previously identified. The evaluation combines objective measurements of system behaviour with subjective feedback collected from users. Together, these perspectives provide a comprehensive assessment of the prototype's effectiveness as an *IEC 61499* development environment.

The evaluation is organised into five complementary areas. Functional validation verifies that the prototype correctly supports the modelling, deployment and monitoring of *IEC 61499* applications. Performance evaluation analyses workflow efficiency and compares key tasks against existing tools. Collaboration evaluation examines the benefits of the real-time collaborative architecture and contrasts it with traditional version control workflows. Usability evaluation focuses on user perception and experience, including a study of existing *IEC 61499* tooling. Finally, the results are discussed in the context of the research objectives and identified limitations.

5.1 Evaluation Goals

The evaluation aims to answer the following questions:

1. Can the proposed solution successfully support the complete development workflow of an *IEC 61499* application?
2. Does the proposed architecture provide advantages over existing desktop based development environments?
3. Does the integration of real-time collaboration improve collaborative development workflows?
4. Do users perceive the proposed solution as accessible, easy to use and an improvement over previous solutions?

To address these questions, the evaluation combined both quantitative and qualitative methods, assessing objective performance characteristics alongside subjective user experience.

5.2 Experimental Setup

A common test application was used throughout the validation process. The same application was implemented using both *PTERO* and *4diac IDE*, allowing direct comparison between environments.

5.2.1 Hardware

The following hardware devices were available and used throughout the evaluation phase of this work:

1. **M1 MacBook Pro** (16GB RAM, 2020): This machine served as the primary development and testing platform for *PTERO*. Being the development machine, it hosted the local development server during iterative testing cycles and was used to evaluate the responsiveness of the collaborative editing interface, the correctness of graph synchronisation and the performance of the *Yjs* based real-time collaboration layer under local conditions.
2. **ASUS TUF Gaming F16** (32GB RAM, 2024): This machine was employed primarily for evaluating the performance of *4diac IDE*.
3. **Raspberry Pi 3** (1GB RAM): This device acted as a permanently running, remotely accessible server throughout the evaluation period. It hosted a persistent instance of the *PTERO* server as well as an instance of *DINASORE*. Its limited memory and processing capacity made it the most representative of real-world target environments and its inclusion in the testing structure was instrumental in establishing the practical boundaries of the collaboration server.

5.2.2 Available Resources

The following resources were made available in the testing environments:

- One *DINASORE* RTE.
- Three function block definitions obtained from the official *DINASORE* tutorial *'Sensorization "Hello World!"'*:
  - `SENSOR_SIMULATOR.fbt` and `SENSOR_SIMULATOR.py`
  - `MOVING_AVERAGE.fbt` and `MOVING_AVERAGE.py`
  - `CONCATENATE_REALS.fbt` and `CONCATENATE_REALS.py`

The selected function blocks provide a representative example of a small *IEC 61499* application while remaining sufficiently simple for controlled testing.

5.2.3 Test Application

The application shown in Figure 5.1 was implemented and deployed during the validation process. It consists of a sensor simulation function block whose output is processed by a moving average filter before being formatted for display.

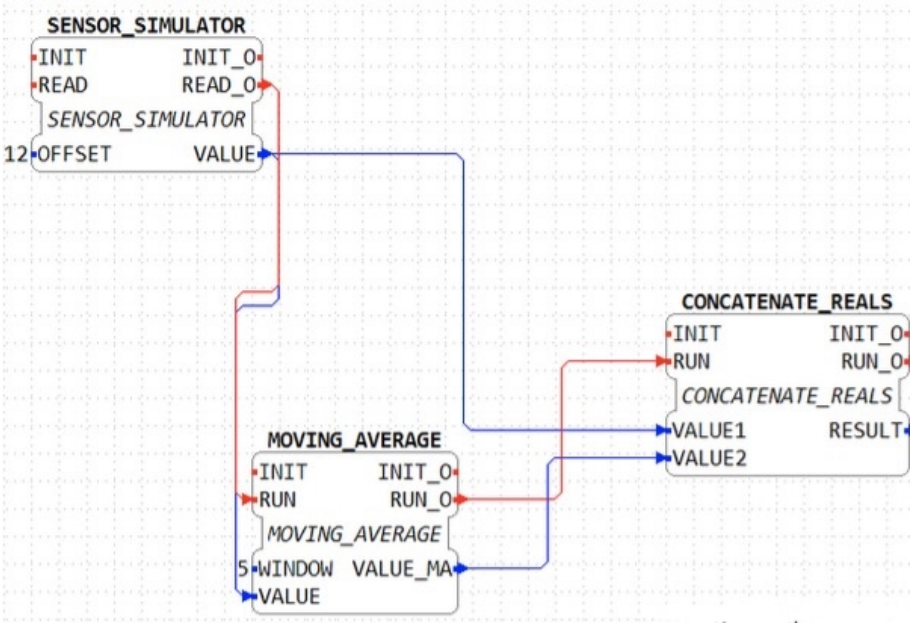


Figure 5.1: Test application used during validation

5.2.4 Expected Behaviour

After deployment, the application should execute correctly on the target *DINASORE* instance and continuously produce output values. These values should be observable through the monitoring facilities provided by the IDE.

5.3 Evaluation Metrics

The evaluation combines both objective and subjective metrics.

5.3.1 Performance Metrics

Performance evaluation considers:

- Installation complexity
- Startup time
- Number of manual configuration steps
- Deployment workflow complexity

5.3.2 Collaboration Metrics

Collaboration capabilities are evaluated through:

- Support for concurrent editing
- Conflict handling mechanisms
- Need for manual synchronisation

5.3.3 Usability Metrics

Usability is assessed through user questionnaires based on:

- Perceived task difficulty
- Interface intuitiveness
- User satisfaction
- Open ended qualitative feedback

5.4 Functional Validation

PTERO was subjected to functional tests to confirm that all core features behave as specified. It was tested by implementing and deploying the test program defined in Section 5.2. As a control, the same was performed in *4diac IDE*. The steps taken were as follows:

1. **Configured available *DINASORE***
   - **4diac IDE**: In the *System Configuration* tab, added a device and set its IP address and the port where *DINASORE* is running, then connected it to the *Ethernet* tunnel.
   - **PTERO**: In the *Config* tab, added a device and set its IP address and the port where *DINASORE* is running, as well as the SSH details of the device which will be used to sync the function blocks files.
2. **Imported the function block files**
   - **4diac IDE**: Copied and pasted the given files to `4diac-ide/typelibrary/`.
   - **PTERO**: Copied and pasted the given files in the *Function Blocks* tab.
3. **Designed the flow**
   - **4diac IDE**: In the *App View*, dragged and dropped the wanted function blocks, then connected the event and data slots. Double-clicked input slots to assign input value.

- **PTERO**: In the *Graph* tab, right-clicked and selected the wanted function blocks to add them, then connected the event and data slots. Right-clicked nodes to assign input value.

4. **Mapped to *DINASORE***

   - **Both**: Right-clicked nodes to map them to the device. They became color coded to show which device they were mapped to.

5. **Synced files with *DINASORE***

   - **4diac IDE**: Called the `synchronize.py` manually, feeding it the needed SSH details.
   - **PTERO**: This was done automatically and without any needed setup when deploying.

6. **Deployed**

   - **Both**: Pressed the *Deploy* button. The sent commands and associated responses were displayed in a console.

7. **Monitored the output**

   - **Both**: The values were displayed on the right side of the output data slots.

Besides verifying that the output values are being monitored correctly, following the *Deployed* step, the commands displayed in the console were directly compared between IDEs. It was verified that both were sending exactly the same commands, the headers as well as the contents were exactly the same and the responses were no different. These results confirmed that the prototype correctly handles the development of *IEC 61499* programs.

While the two workflows are broadly equivalent in the number of steps required, they differ in several procedural details. Configuring a device in *PTERO* requires additional SSH credentials upfront, a step absent in *4diac IDE*, however, this overhead is offset later in the workflow, as file synchronisation with *DINASORE* is a manual command line operation in *4diac IDE*, while it is handled automatically by *PTERO* at deploy time. Similarly, importing function block files is more straightforward in *PTERO*, which accepts files directly through the interface, whereas *4diac IDE* requires manual placement into a specific directory in the filesystem. These differences reflect a deliberate design philosophy in *PTERO* of consolidating configuration complexity into the tool itself.

5.5 Performance Evaluation

This solution not only aims to be able to do the same workload as *4diac IDE*, but also have capabilities and an inherent design that justify its existence. Therefore, this section compares the performance of several real world tasks between the IDEs. The listed tasks are the ones that

demonstrated to be significantly different between solutions, others were analysed but proved to not be worth the discussion.

- **Install**
  - **4diac IDE**: Install a desktop app. There is no compatible version for macOS that supports *DINASORE*.
  - **PTERO**: Install a *Node.js* server. By working in a web browser, it is crossplatform without much development effort, which allows macOS users to use it. The downside is that it is not as straightforward to install for just local development.
- **Open**
  - **4diac IDE**: Average of 4 seconds to boot the application in the *ASUS TUF Gaming F16*. The startup overhead is inherent to the *Eclipse* based desktop architecture underlying this tool.
  - **PTERO**: Average of less than a second to load webpage. The use of lightweight libraries and the move of heavy workloads to the server allow the page to load in an instant.
- **Create and edit function block files**
  - **4diac IDE**: While it does have a way to create and edit the *.fbt* files inside the application, there is no built in *Python* text editor.
  - **PTERO**: Has a text editor for both types of files.
- **Collaboration**
  - **4diac IDE**: An external periodic version control system like *Git* needs to be used. Structured data in formats like *XML* is not handled properly by these types of version control system, which results in frequent merge conflicts [2]. The following test was conducted:
    1. A project has a graph with 1 function block added. 2 users are working on the graph.
    2. User #1 deletes the function block.
    3. User #2 modifies the position of the function block.
    4. User #1 commits and pushes.
    5. User #2 tries to pull the changes but gets a merge conflict
  - **PTERO**: Has a realtime collaboration system. Any change made is immediately propagated to every client and resolved by a CRDT, ensuring no conflicts need to be handled manually and damage is minimised.
- **Synchronising files with the *DINASOREs***

  - **4diac IDE**: Required to manually use an external script before deployment.
  - **PTERO**: The server itself will connect and synchronise the files with the *DINASOREs*. This happens automatically when attempting to deploy, ensuring that deployments never fail due to the human error of forgetting to sync the files.

- **Remote deployment**

  - **4diac IDE**: Every IDE must have a valid connection with each available *DINASORE*, as well as *SSH* access. This gets increasingly complicated if the user wants to make a deployment while in a different network.
  - **PTERO**: By having the server as the middleware, only it must be exposed for the users to connect. With only the server permanently having direct connection with the *DINASORE* devices, no individual handling is needed.

5.6 Collaboration Evaluation

Nicolaescu et al. claim that *Yjs* is able to handle an average of 100 actions per millisecond [16] which is obviously good enough for this use case, as it does not require millisecond precision. Taking connection with the remote server into account, a test was devised to understand how well the *Raspberry Pi* could handle multiple users at the same time. A script was created that initialised a set number of clients, connected them to the remote server and sent a change that would get propagated across the clients. By measuring the time that would take for the clients to receive the change, the chart in Figure 5.2 was obtained. The results demonstrated that even a resource constrained machine accessed over a network can serve as an adequate collaboration server for a small team, however, the *Raspberry Pi* deployment was found to be unsuitable beyond approximately 240 simultaneous clients, at which point the hardware prevented additional connections from being established reliably.

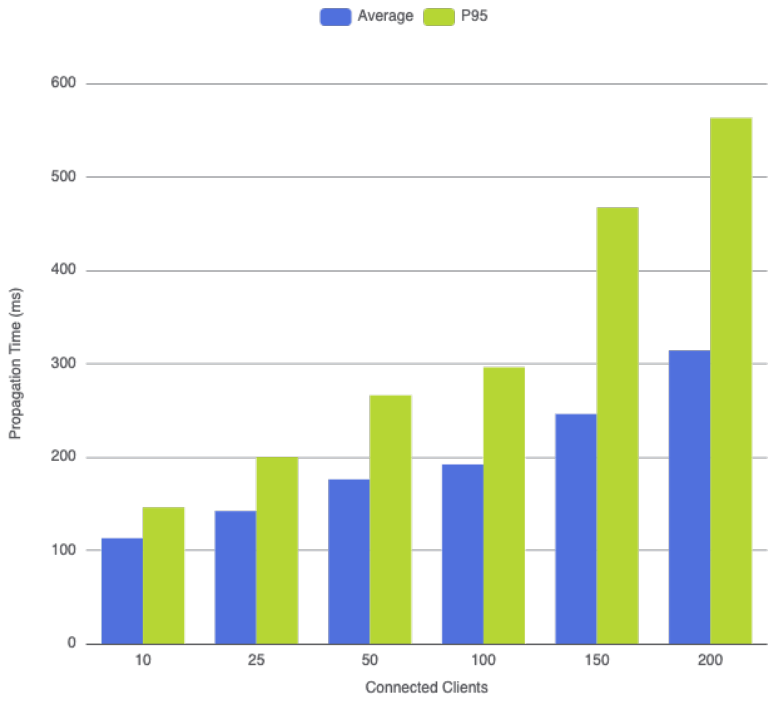


Figure 5.2: Performance results of *Yjs* running in the *Raspberry Pi*

The same test was performed with a server running on the *M1 MacBook Pro* and the clients also in the same machine, removing any delay resulting from network delay and in a much more powerful machine. It was possible to connect 3000 clients with an average propagation time of 65 ms and a P95 of 221 ms, proving that if there is a need for hundreds of clients to be connected at once, the solution is able to scale with the hardware to accommodate it.

5.7 Usability Evaluation

Two questionnaires were designed and given to participants. One focused on the user experience of working with *4diac IDE* as the underlying development environment for *IEC 61499*. The other asked both for a usability opinion on *PTERO*, as well as a direct comparison between these two IDEs. The participants of the questionnaires were:

- 11 Students from Master in Electrical and Computer Engineering (MEEC) at Faculdade de Engenharia da Universidade to Porto (FEUP), enrolled in the course of Information Systems (SINF), in which, over the course of a semester, they were tasked with using *4diac IDE* for modelling FBD programs with *DINASORE* as the RTE. Having completed a full semester of work on this environment, these students possessed sufficient practical familiarity with the tool to provide informed usability assessments and represent a realistic profile of future *PTERO* adopters in academic and educational contexts.

- 5 Students at FEUP, working on Master's dissertations at Research Center for Systems and Technologies (SYSTEC) on topics directly related to *4diac IDE* and *DINASORE*. Their deeper technical engagement with the ecosystem made them particularly well positioned to evaluate the correctness and completeness of *PTERO*'s workflow.

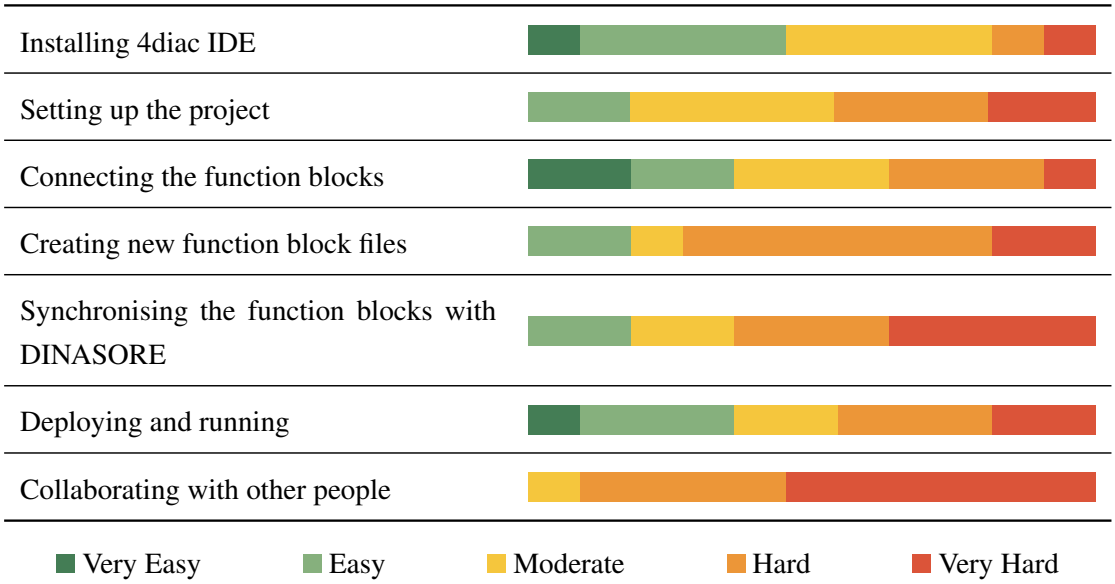
5.7.1 4diac IDE Usability

A usabilty study on *4diac IDE* has already been conducted by Weber, Zoitl, and HuBmann [27], but given as the this new proposed solution introduces new concepts and expectations, a new usability study must be done to understand how this tool fairs in the specific fields that *PTERO* sets out to innovate in.

Difficulty

With the use of a Likert [13] table, the difficulty of doing several different tasks using this IDE was analysed. Ranging from *Very Easy* to *Very Hard*, this section had the intent of understanding how easy and straightforward each step of *IEC 61499* development was. If a specific step was difficult in *4diac IDE*, it will be interesting to see how it compares with *PTERO*. The results are displayed in Table 5.1.

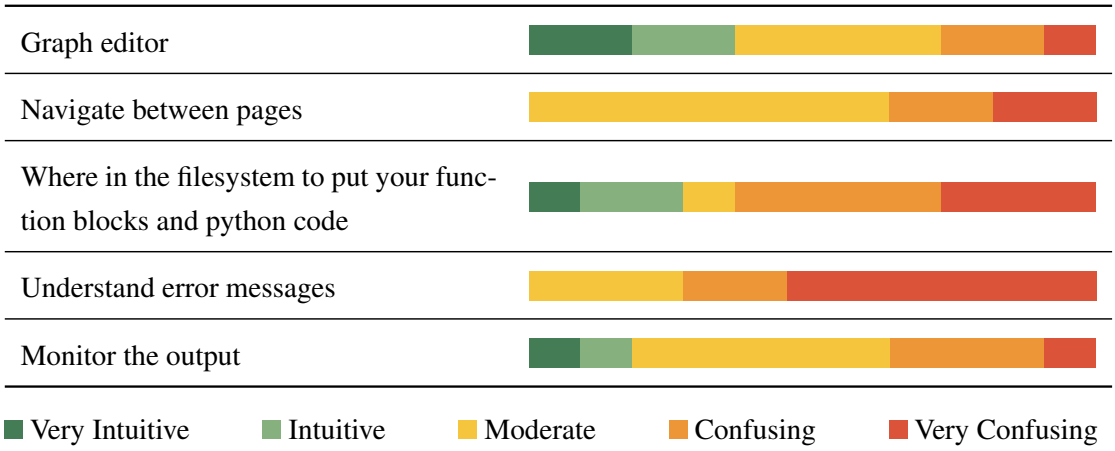
Table 5.1: Perceived difficulty of tasks in *4diac IDE*



Intuitiveness

Next, another Likert table, this time on how intuitive it was to understand each part of the IDE. This way the design decisions of *4diac IDE* can be critically analysed. The results are displayed in Table 5.2.

Table 5.2: Perceived intuitiveness of key parts of *4diac IDE*



Open Feedback

The questionnaire concluded with open-ended questions that provided additional insight into the user experience of working with *4diac IDE*.

What did you like most about using 4diac IDE? The most frequently mentioned positive aspect was the modularity of programming with function blocks, which made the overall structure of programs easier to understand.

> "The files were simple to implement in .fbt and python, so the logic could be easily done prior and then sequentially implement our ideas in code and then route them."

> "I highly appreciated the modularity of the system, specifically the ability to design independent data manipulation pipelines for each individual machine. This decentralised approach feels highly relevant and closely mimics real-world industrial scenarios."

What were the biggest challenges you faced? The most common difficulties were related to deployment, management of function block files and collaboration. Participants reported confusion regarding where files needed to be placed in the filesystem and how synchronisation with *DINASORE* should be performed. As SINF students had been tasked over the semester to work in groups, the collaboration issues were a recurring theme in their answers. It is also noteworthy that the average response length increased substantially, from 108 characters in the previous question to 510 characters in this one. This suggests that participants had considerably more detailed feedback when discussing the challenges, often providing detailed explanations and examples.

> "Collaborating with colleagues was very hard, because of the management of file versions. Only one could be working at a time and we could only change the file after it has uploaded on Github."

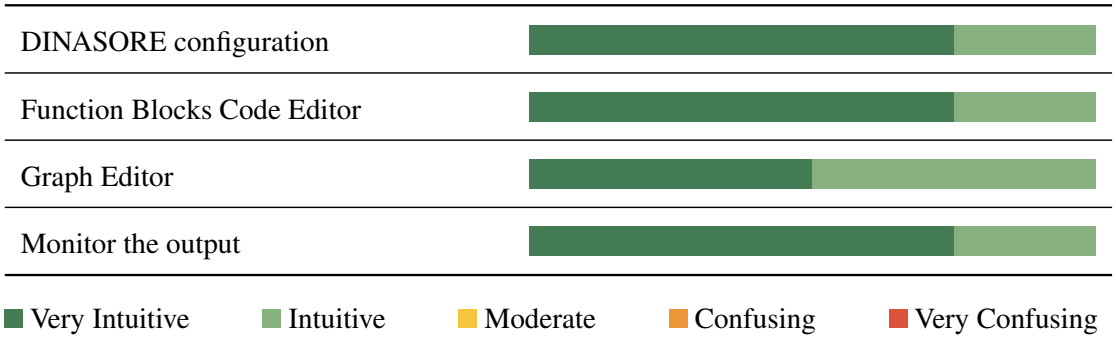
> "Connecting to DINASORE and understanding that there are changes to make in both file systems."

> "(...) Vague Error Messages: Debugging can be quite difficult due to a lack of specificity in error messages. (...) 4diac IDE occasionally rejects the deployment of a project without explaining exactly why, sometimes only providing the name of an input variable without further context."

5.7.2 PTERO Usability

Intuitiveness

Table 5.3: Perceived intuitiveness of key parts of *PTERO*

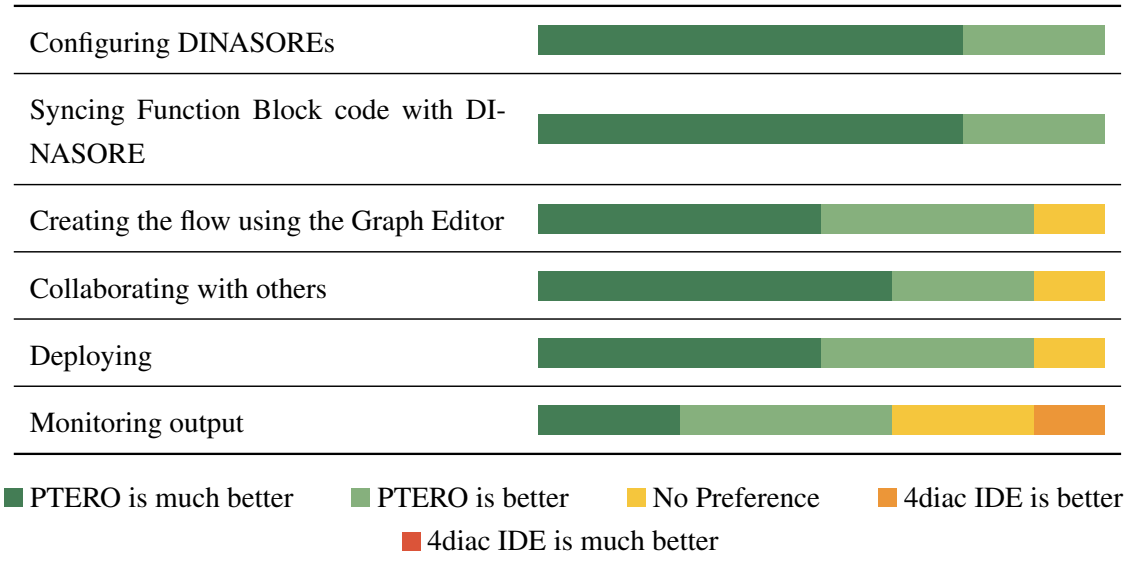


System Usability Scale

Score: 88,75

5.7.3 Comparison

Table 5.4: Direct comparison of usability of *PTERO* and *4diac IDE*



Open Feedback

What do you like/dislike about a remote web-based solution like PTERO when compared to a desktop app like 4diac IDE? Responses to this question were predominantly positive. Participants highlighted the reduced setup requirements, improved accessibility and simpler user interface provided by the web-based approach. Several respondents also noted that integrating the `.fbt` and `.py` editors into a single environment simplified the development workflow. While some usability improvements were suggested, participants considered PTERO easier to use than *4diac IDE*.

> "Easier to setup for group work. Accessible from any PC."
>
> "PTERO seems easier to setup and overall use. Loved the feature of writing the .fbt and .py files on the same tab."
>
> "PTERO is an easy to use and very intuitive platform that surpasses in user friendliness the 4DIAC IDE when compared to its complexity and function block management."

5.8 Discussion

The evaluation results indicate that the proposed solution successfully achieves functional parity with the established *4diac IDE* while introducing architectural and usability improvements that address several of the limitations identified in traditional *IEC 61499* development environments.

The functional validation confirmed that *PTERO* correctly supports the complete development workflow, including configuration, modelling, deployment and monitoring of *IEC 61499* applications. Both systems generated identical deployment commands and produced equivalent runtime behaviour in *DINASORE*. This equivalence is an important result, as it demonstrates that the proposed abstraction layer does not compromise correctness or compatibility with existing tooling. Instead, it preserves interoperability while offering a different interaction model.

The performance evaluation further complements these findings by highlighting how architectural decisions in *PTERO* translate into measurable workflow improvements. While both systems are capable of executing equivalent *IEC 61499* applications, *PTERO* reduces the number of manual steps required across key development tasks such as project setup, synchronisation and deployment. A central factor behind this improvement is the tighter integration between the different components of the development pipeline. In *4diac IDE*, the development workflow is distributed across multiple loosely connected tools and processes, requiring explicit user actions to maintain consistency between the modelling environment, local file system and remote runtime. In contrast, *PTERO* internalises these responsibilities within the platform itself, allowing project state, file management and deployment logic to be handled automatically as part of a single unified system.

The usability questionnaires provide strong evidence that a web-based implementation can offer a significantly improved user experience compared to *4diac IDE*. Participants consistently reported higher levels of perceived ease of use, intuitiveness and overall satisfaction when interacting with *PTERO*. This improvement can be attributed not only to interface design choices, but also to the inherent advantages of web technologies in this context.

Furthermore, it was observed that the integration of all development tasks into a single unified interface contributes to a reduction in cognitive load. Unlike *4diac IDE*, where users must switch between multiple tools and external processes (such as file system management and synchronisation scripts), the web-based environment consolidates modelling, editing, deployment and monitoring into a coherent workflow. The results also suggest that responsiveness and perceived simplicity play an important role in user satisfaction.

Overall, the findings indicate that the adoption of web technologies is not merely a change in deployment platform, but a design decision that directly influences usability by simplifying access, unifying workflows and reducing structural complexity for end users.

Chapter 6

Discussion

Chapter 7

Conclusion

References

[1] *4diac IDE Website*. URL: https://eclipse.dev/4diac/4diac_ide/.

[2] E. Axelsson and Sérgio Batista. "Version Control of structured data: A study on different approaches in XML". In: 2015. URL: https://www.semanticscholar.org/paper/Version-Control-of-structured-data%3A-A-study-on-in-Axelsson-Batista/323392e2fee60020d73f949fc996c31bf8c5b7ee (visited on 05/31/2026).

[3] Zakaria Bencherki et al. "IEC 61499: A Key Enabler for Distributed Automation in Industry 4.0". In: *2024 International Conference on Electrical, Computer and Energy Technologies (ICECET*. July 2024, pp. 1–6. DOI: 10.1109/ICECET61485.2024.10698592. URL: https://ieeexplore.ieee.org/document/10698592 (visited on 06/11/2026).

[4] Young-Jo Cho et al. "Function Block Diagram Approach for Manufacturing Process Control Programming". en. In: *IFAC Proceedings Volumes* 26.2 (July 1993), pp. 461–464. ISSN: 14746670. DOI: 10.1016/S1474-6670(17)48510-7. URL: https://linkinghub.elsevier.com/retrieve/pii/S1474667017485107 (visited on 01/28/2026).

[5] *Editor Bindings | Yjs Docs*. en. Dec. 2023. URL: https://docs.yjs.dev/ecosystem/editor-bindings (visited on 06/09/2026).

[6] Juliane Fischer et al. "Measuring the Overall Complexity of Graphical and Textual IEC 61131-3 Control Software". In: *IEEE Robotics and Automation Letters* 6.3 (July 2021), pp. 5784–5791. ISSN: 2377-3766, 2377-3774. DOI: 10.1109/LRA.2021.3084886. URL: https://ieeexplore.ieee.org/document/9444196/ (visited on 06/07/2026).

[7] Francisco Folgado et al. "Review of Industry 4.0 from the Perspective of Automation and Supervision Systems: Definitions, Architectures and Recent Trends". en. In: *Electronics* 13.4 (Feb. 2024), p. 782. ISSN: 2079-9292. DOI: 10.3390/electronics13040782. URL: https://www.mdpi.com/2079-9292/13/4/782 (visited on 06/11/2026).

[8] Marcelo V. Garcia et al. "Engineering tool to develop CPPS based on IEC-61499 and OPC UA for oil&gas process". en. In: *2017 IEEE 13th International Workshop on Factory Communication Systems (WFCS)*. Trondheim, Norway: IEEE, May 2017, pp. 1–9. ISBN: 978-1-5090-5788-7. DOI: 10.1109/WFCS.2017.7991969. URL: http://ieeexplore.ieee.org/document/7991969/ (visited on 01/30/2026).

[9] *IEC 61131-3*. en. Page Version ID: 1339669093. Feb. 2026. URL: https://en.wikipedia.org/w/index.php?title=IEC_61131-3&oldid=1339669093 (visited on 05/11/2026).

[10] *IEC 61499 Wikipedia*. URL: https://en.wikipedia.org/wiki/IEC_61499.

[11] Steven Kelly. "Collaborative Modelling with Version Control". In: ed. by Martina Seidl and Steffen Zschaler. Vol. 10748. Book Title: Software Technologies: Applications and Foundations Series Title: Lecture Notes in Computer Science. Cham: Springer International Publishing, 2018, pp. 20–29. ISBN: 978-3-319-74729-3 978-3-319-74730-9. DOI: 10.1007/978-3-319-74730-9_3. URL: http://link.springer.com/10.1007/978-3-319-74730-9_3 (visited on 06/04/2026).

[12] Martin Kleppmann and Alastair R. Beresford. "A Conflict-Free Replicated JSON Datatype". In: *IEEE Transactions on Parallel and Distributed Systems* 28.10 (Oct. 2017), pp. 2733–2746. ISSN: 1045-9219. DOI: 10.1109/TPDS.2017.2697382. URL: http://ieeexplore.ieee.org/document/7909007/ (visited on 06/09/2026).

[13] R. Likert. "A TECHNIQUE FOR THE MEASUREMENT OF ATTITUDE SCALES". In: 1932. URL: https://www.semanticscholar.org/paper/A-TECHNIQUE-FOR-THE-MEASUREMENT-OF-ATTITUDE-SCALES-Likert/c7c4b379b2ca0d0c134bc3c0f114a326e (visited on 06/01/2026).

[14] Tuojian Lyu et al. "Towards Web Platform for Cloud-Based Virtual Commissioning of IEC 61499 Distributed Automation Systems". en. In: *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*. Padova, Italy: IEEE, Sept. 2024, pp. 1–4. ISBN: 979-8-3503-6123-0. DOI: 10.1109/ETFA61755.2024.10710351. URL: https://ieeexplore.ieee.org/document/10710351/ (visited on 12/17/2025).

[15] László Monostori. "Cyber-physical Production Systems: Roots, Expectations and R&D Challenges". en. In: *Procedia CIRP* 17 (2014), pp. 9–13. ISSN: 22128271. DOI: 10.1016/j.procir.2014.03.115. URL: https://linkinghub.elsevier.com/retrieve/pii/S2212827114003497 (visited on 01/28/2026).

[16] Petru Nicolaescu et al. "Yjs: A Framework for Near Real-Time P2P Shared Editing on Arbitrary Data Types". en. In: ed. by Philipp Cimiano et al. Vol. 9114. Book Title: Engineering the Web in the Big Data Era Series Title: Lecture Notes in Computer Science. Cham: Springer International Publishing, 2015, pp. 675–678. ISBN: 978-3-319-19889-7 978-3-319-19890-3. DOI: 10.1007/978-3-319-19890-3_55. URL: http://link.springer.com/10.1007/978-3-319-19890-3_55 (visited on 06/09/2026).

[17] Linna Pang et al. "Formal verification of function blocks applied to IEC 61131-3". en. In: *Science of Computer Programming* 113 (Dec. 2015), pp. 149–190. ISSN: 01676423. DOI: 10.1016/j.scico.2015.10.005. URL: https://linkinghub.elsevier.com/retrieve/pii/S0167642315002981 (visited on 01/29/2026).

[18] E. Pereira, João C. P. Reis, and G. Gonçalves. "DINASORE: A Dynamic Intelligent Reconfiguration Tool for Cyber-Physical Production Systems". In: 2020. URL: https://www.semanticscholar.org/paper/DINASORE%3A-A-Dynamic-Intelligent-Reconfiguration-for-Pereira-Reis/8337325c710a6f08eb9fc85855852bc8aaa2e9d1 (visited on 06/02/2026).

[19] Gonçalo Pinto et al. "Digital Factory: An Industrial Case Study for Function Block-Oriented Development". en. In: *2023 IEEE 9th World Forum on Internet of Things (WF-IoT)*. Aveiro, Portugal: IEEE, Oct. 2023, pp. 01–06. ISBN: 979-8-3503-1161-7. DOI: 10.1109/WF-IoT58464.2023.10539486. URL: https://ieeexplore.ieee.org/document/10539486/ (visited on 01/29/2026).

[20] Oana Rohat and Dan Popescu. “Remote web-based execution of IEC 61499 function blocks”. en. In: *Proceedings of the 2014 6th International Conference on Electronics, Computers and Artificial Intelligence (ECAI)*. Bucharest, Romania: IEEE, Oct. 2014, pp. 33–38. ISBN: 978-1-4799-5478-0 978-1-4799-5479-7. DOI: 10.1109/ECAI.2014.7090220. URL: http://ieeexplore.ieee.org/document/7090220/ (visited on 01/28/2026).

[21] Oliver Schmid, Agnes Lisowska Masson, and Béat Hirsbrunner. “Real-time collaboration through web applications: an introduction to the Toolkit for Web-based Interactive Collaborative Environments (TWICE)”. en. In: *Personal and Ubiquitous Computing* 18.5 (June 2014), pp. 1201–1211. ISSN: 1617-4909, 1617-4917. DOI: 10.1007/s00779-013-0729-0. URL: http://link.springer.com/10.1007/s00779-013-0729-0 (visited on 06/04/2026).

[22] Radimir Sorokin, Sandeep Patil, and Valeriy Vyatkin. “Novel development tool for IEC 61499 based on domain-specific languages”. en. In: *IFAC-PapersOnLine* 55.2 (2022), pp. 439–444. ISSN: 24058963. DOI: 10.1016/j.ifacol.2022.04.233. URL: https://linkinghub.elsevier.com/retrieve/pii/S2405896322002348 (visited on 01/28/2026).

[23] Radimir Sorokin and Valeriy Vyatkin. “Towards web-applications for domain-specific languages and development tools”. In: *2023 IEEE 2nd Industrial Electronics Society Annual On-Line Conference (ONCON)*. Dec. 2023, pp. 1–6. DOI: 10.1109/ONCON60463.2023.10431381. URL: https://ieeexplore.ieee.org/document/10431381 (visited on 06/13/2026).

[24] Tomás Torres, Gil Gonçalves, and Rui Pinto. “Literature Review on Cloud-Based Service-Oriented Architecture for IEC 61499 Distributed Control Systems:” en. In: *Proceedings of the 15th International Conference on Cloud Computing and Services Science*. Porto, Portugal: SCITEPRESS - Science and Technology Publications, 2025, pp. 174–181. ISBN: 978-989-758-747-4. DOI: 10.5220/0013293400003950. URL: https://www.scitepress.org/DigitalLibrary/Link.aspx?doi=10.5220/0013293400003950 (visited on 01/28/2026).

[25] *VSCode Web*. URL: https://code.visualstudio.com/docs/setup/vscode-web.

[26] Valeriy Vyatkin and Julien Chouinard. *On comparisons of the ISaGRAF implementation of IEC 61499 with FBDK and other implementations*. Journal Abbreviation: 6th IEEE International Conference on Industrial Informatics Pages: 294 Publication Title: 6th IEEE International Conference on Industrial Informatics. Aug. 2008. ISBN: 978-1-4244-2170-1. DOI: 10.1109/INDIN.2008.4618111.

[27] Thomas Weber, Alois Zoitl, and Heinrich HuBmann. “Usability of Development Tools: A CASE-Study”. en. In: *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. Munich, Germany: IEEE, Sept. 2019, pp. 228–235. ISBN: 978-1-7281-5125-0. DOI: 10.1109/MODELS-C.2019.00037. URL: https://ieeexplore.ieee.org/document/8904489/ (visited on 01/30/2026).

[28] *Zed Roadmap*. URL: https://zed.dev/roadmap.

[29] Alois Zoitl, Thomas Strasser, and Gerhard Ebenhofer. “Developing modular reusable IEC 61499 control applications with 4DIAC”. en. In: *2013 11th IEEE International Conference on Industrial Informatics (INDIN)*. Bochum, Germany: IEEE, July 2013, pp. 358–363. ISBN: 978-1-4799-0752-6. DOI: 10.1109/INDIN.2013.6622910. URL: http://ieeexplore.ieee.org/document/6622910/ (visited on 01/29/2026).

Appendix A

Implementation

A.1 Function Block Files

```
<FBType Name="NEW_FB_1">
  <InterfaceList>
    <EventInputs>
      <Event Name="INIT" Type="Event"/>
      <Event Name="READ" Type="Event">
        <With Var="VARIABLE_1"/>
        <With var="VARIABLE_2"/>
        <With var="VARIABLE_3"/>
      </Event>
    </EventInputs>
    <EventOutputs>
      <Event Name="INIT_O" Type="Event"/>
      <Event Name="READ_O" Type="Event">
        <With Var="DATA_1"/>
      </Event>
    </EventOutputs>
    <InputVars>
      <VarDeclaration Name="VARIABLE_1" Type="STRING"/>
      <VarDeclaration Name="VARIABLE_2" Type="REAL"/>
      <VarDeclaration Name="VARIABLE_3" Type="INT"/>
    </InputVars>
    <OutputVars>
      <VarDeclaration Name="DATA_1" Type="STRING"/>
    </OutputVars>
  </InterfaceList>
</FBType>
```

Listing A.1: Example *xml* definition of a function block

```
class NEW_FB_1:
  def __init__(self):
    pass

```

```
  def schedule(self, event_name, event_value, variable_1, variable_2, variable_3):
    if event_name == "INIT":
      return [event_value, None, ""]

    elif event_name == "READ":
      return [None, event_value, ""]
```

Listing A.2: Example *Python* code of a function block

A.2 Synchronisation

```
{
  master-fbs-path: 'sync/fbs',
  dinasores: [
    {
      address: 'x.x.x.x',
      port: 22,
      username: 'root',
      password: 'dinasore',
      dinasore-path: '/root/dinasore'
    },
    {
      address: 'y.y.y.y',
      port: 22,
      username: 'root',
      password: 'dinasore',
      dinasore-path: '/root/dinasore'
    }
  ]
}
```

Listing A.3: Sync configuration

A.3 Communication

```
def build_message(message_payload, configuration_name):
  # build the first part of the header
  hex_input = '{:04x}'.format(len(configuration_name))
  second_byte = int(hex_input[0:2], 16)
  third_byte = int(hex_input[2:4], 16)
  response_len = struct.pack('BB', second_byte, third_byte)
  response_header_1 = b''.join([b'\x50', response_len])

  # build the second part of the header
  hex_input = '{:04x}'.format(len(message_payload))
  second_byte = int(hex_input[0:2], 16)
```

```
  third_byte = int(hex_input[2:4], 16)
  response_len = struct.pack('BB', second_byte, third_byte)
  response_header_2 = b''.join([b'\x50', response_len])

  # join the 3 parts
  response = b''.join([response_header_1, configuration_name, response_header_2,
      message_payload])
  return response
```

Listing A.4: build_message() in Python

```
function build_message(message_payload, configuration_name) {
  const configBuffer = Buffer.from(configuration_name, 'utf-8');
  const messageBuffer = Buffer.from(message_payload, 'utf-8');

  // Header 1: [0x50][len_hi][len_lo]
  const header1 = Buffer.alloc(3);
  header1[0] = 0x50;
  header1.writeUInt16BE(configBuffer.length, 1);

  // Header 2: [0x50][len_hi][len_lo]
  const header2 = Buffer.alloc(3);
  header2[0] = 0x50;
  header2.writeUInt16BE(messageBuffer.length, 1);

  return Buffer.concat([
    header1,
    configBuffer,
    header2,
    messageBuffer
  ]);
}
```

Listing A.5: build_message() in JavaScript

A.4 Send Messages

First we need to send the two commands in Listing A.6 to every device. A `QUERY` request and a `CREATE` request that will create a *EMB_RES* resource which will be automatically connected to the *INIT* port of all other function block nodes to initialize them. Both these commands will have an empty configuration name, as *EMB_RES* has not been created yet and therefore cannot be bounded to.

```
<Request Action="QUERY" ID="0">
  <FB Name="*" Type="*"/>
</Request>

```

```
<Request Action="CREATE" ID="1">
  <FB Name="EMB_RES" Type="EMB_RES"/>
</Request>
```

Listing A.6: Setup deployment commands

Next, depending on which commands are mapped to each machine, we `CREATE` all the mapped nodes and `WRITE` the initial input values with the commands in Listing A.7. The configuration name in these and all following commands will be "*EMB_RES*" as they need to be bound to it in order to be initialised and ran.

```
<Request Action="CREATE" ID="2">
  <FB Name="SENSOR_SIMULATOR" Type="SENSOR_SIMULATOR"/>
</Request>

<Request Action="WRITE" ID="3">
  <Connection Destination="SENSOR_SIMULATOR.OFFSET" Source="12"/>
</Request>

<Request Action="CREATE" ID="4">
  <FB Name="MOVING_AVERAGE" Type="MOVING_AVERAGE"/>
</Request>

<Request Action="WRITE" ID="5">
  <Connection Destination="MOVING_AVERAGE.WINDOW" Source="5"/>
</Request>
```

Listing A.7: `CREATE` nodes deployment commands

Then the connections are created between the mapped nodes. A `Source` a `Destination` must be provided, with the format `"NODE_NAME.SLOT_NAME"`, using the commands in Listing A.8.

```
<Request Action="CREATE" ID="6">
  <Connection Destination="SENSOR_SIMULATOR.READ_0" Source="MOVING_AVERAGE.RUN"/>
</Request>

<Request Action="CREATE" ID="7">
  <Connection Destination="SENSOR_SIMULATOR.VALUE" Source="MOVING_AVERAGE.VALUE"/>
</Request>
```

Listing A.8: `CREATE` links deployment commands

Finally the `START` command in Listing A.9 can be issued, which will request for the start of the program execution. For this command the configuration name used will also be "*EMB_RES*" as it is this resource that will begin the execution of the program.

```
1 <Request Action="START" ID="8"/>
```

Listing A.9: `START` deployment command

After each request is sent, a response will be returned by the *DINASORE*. It is important for the IDE to listen for these responses and only send the next request once the response corresponding to the previous request is received. The responses have the format shown in Listing A.10. The `ID` of the response will be the same of the associated request. The format of the responses is the same as the requests and the configuration name that is used is empty.

```
1 <Response ID="8"/>
```

Listing A.10: `Response` after a deployment command is sent

A.5 Watch

Note in Listing A.11 that the `ID` is reset after deploying.

```
1 <Request ID="0" Action="CREATE">
2   <Watch Source="SENSOR_SIMULATOR.VALUE" Destination="*" />
3 </Request>
4
5 <Request ID="1" Action="CREATE">
6   <Watch Source="MOVING_AVERAGE.VALUE_MA" Destination="*" />
7 </Request>
```

Listing A.11: `CREATE` watches after deploying

After creating the watches we can send a request to receive the values using the command in Listing A.12. This will return the response in Listing A.13, which by parsing the values, allows the prototype to display them in the *Graph View*.

```
1 <Request ID="2" Action="READ">
2   <Watches/>
3 </Request>
```

Listing A.12: Request to `READ` watches

```
1 <Response ID="2">
2   <Watches>
3     <Resource name="EMB_RES">
```

```
4       <FB name="SENSOR_SIMULATOR">
5         <Port name="VALUE">
6           <Data value="13.75" />
7         </Port>
8       </FB>
9       <FB name="MOVING_AVERAGE">
10        <Port name="VALUE">
11          <Data value="13.3" />
12        </Port>
13      </FB>
14    </Resource>
15  </Watches>
16 </Response>
```

Listing A.13: Response to `READ` watches